

Digital Design (SET 111)

L10

Functions of Combinational Logic

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.



- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Application
- ☐ Decoder
- ☐ Encoder
- ☐ Application
- ☐ Multiplexer
- ☐ De-multiplexer

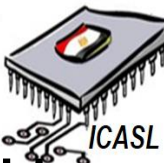


- ☐ **Half and Full Adders**
- ☐ **Parallel Binary Adders**
- ☐ **Binary Subtractor**
- ☐ **Parallel Binary Adder/Subtractor**
- ☐ **Application**
- ☐ **Decoder**
- ☐ **Encoder**
- ☐ **Application**
- ☐ **Multiplexer**
- ☐ **De-multiplexer**

Half and Full Adders

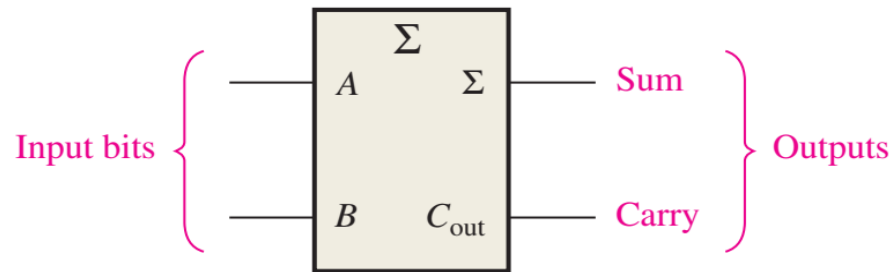


- ☐ **Adders** are important in computers and also in other types of **digital** systems.
- ☐ An **understanding** of the basic adder operation is fundamental to the study of digital systems.
- ☐ After completing this section, you should be able to
 - ☐ **Describe** the function of a **half-adder**
 - ☐ **Draw** a **half-adder** logic diagram
 - ☐ Describe the function of **the full-adder**
 - ☐ Draw a **full-adder** logic diagram using **half-adders**
 - ☐ Implement a **full-adder** using AND-OR logic



Half Adders

- The **half-adder** accepts **two** binary digits on its inputs and produces **two** binary digits on its outputs → a sum bit and a carry bit.



Logic symbol

Truth Table

A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Σ = sum

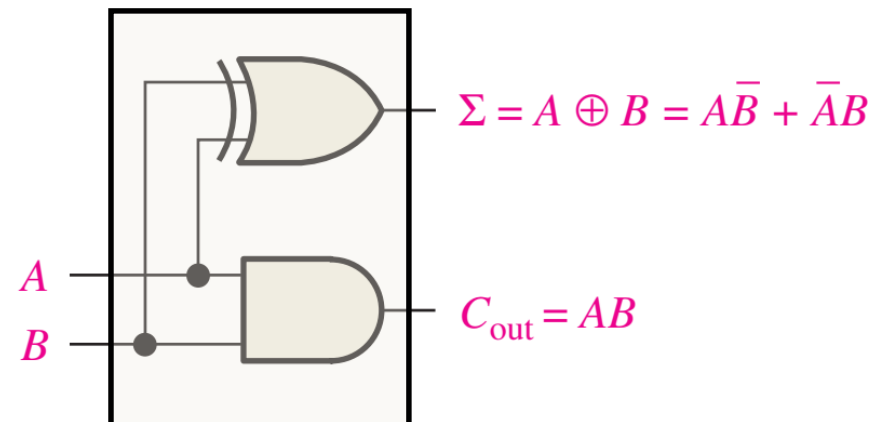
C_{out} = output carry

		B	
		0	1
A	0	0	1
	1	1	0

$$\text{Sum} = \overline{A} \cdot B + A \cdot \overline{B} = A \oplus B$$

		B	
		0	1
A	0	0	0
	1	0	1

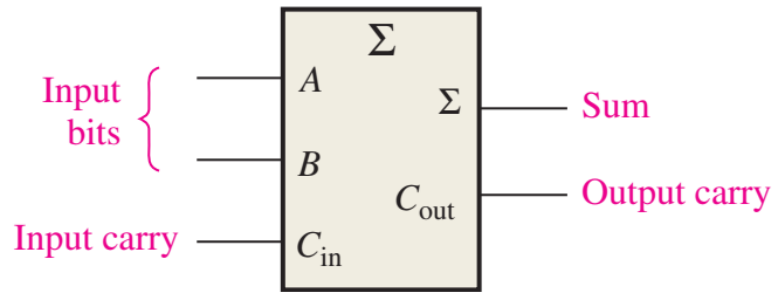
$$C_{out} = A \cdot B$$



Half-adder logic diagram

Full Adders

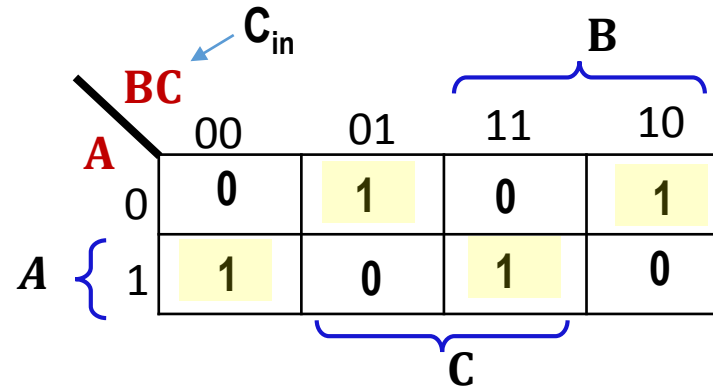
- ❑ The **full-adder** accepts three bits (**two** input bits and an **input carry**) and generates a **sum** output and an output carry. The basic difference between a **full-adder** and a **half-adder** is that the full-adder accepts an input carry.



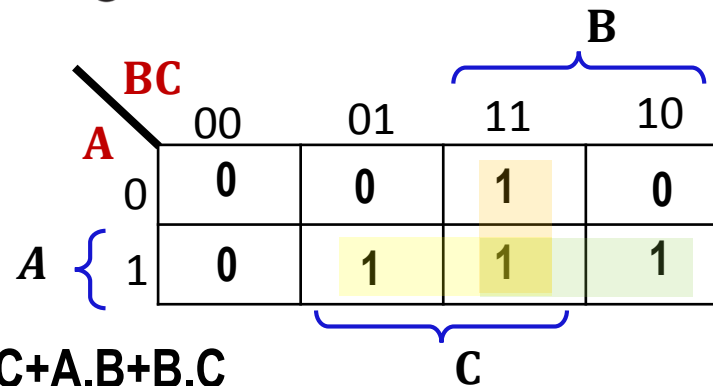
Logic symbol

Truth Table

A	B	C _{in}	C _{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$\begin{aligned}
 \text{Sum} &= A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot \overline{C} + \overline{A} \cdot B \cdot C \\
 &= A \cdot (\overline{B} \cdot \overline{C} + B \cdot C) + \overline{A} \cdot (\overline{B} \cdot C + B \cdot \overline{C}) \\
 &= A \cdot (\overline{B} \oplus \overline{C}) + \overline{A} \cdot (B \oplus C) \\
 &= A \oplus B \oplus C
 \end{aligned}$$



$$C_{\text{out}} = A \cdot C + A \cdot B + B \cdot C$$

$$C_{\text{out}} = A \cdot \overline{B} \cdot C + A \cdot B \cdot C + \overline{A} \cdot B \cdot C + A \cdot B \cdot \overline{C} = A \cdot B + (A \oplus B) \cdot C_{\text{in}}$$

Note:

XOR Gate

$$A \oplus B = \overline{A} \cdot B + A \cdot \overline{B}$$

XNOR Gate

$$A \odot B = \overline{A} \cdot \overline{B} + A \cdot B$$

$$A \odot B = \overline{A \oplus B}$$

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}$$

$$\begin{aligned}
 C_{\text{out}} &= A \cdot B + A \cdot C_{\text{in}} + B \cdot C_{\text{in}} \\
 &= A \cdot B + (A \oplus B) \cdot C_{\text{in}}
 \end{aligned}$$

Full Adders



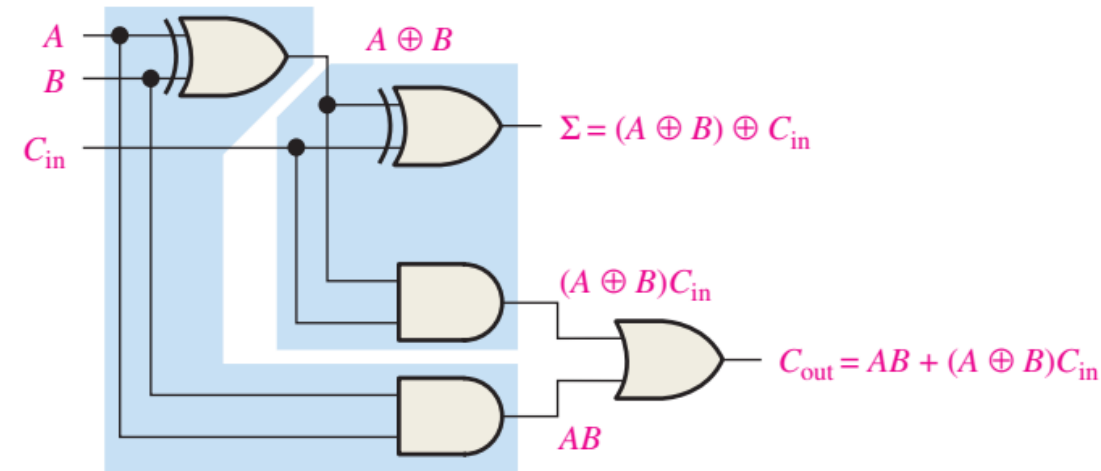
- Full adder implementation using (a) logic circuit and (b) two- half adders

Full adder

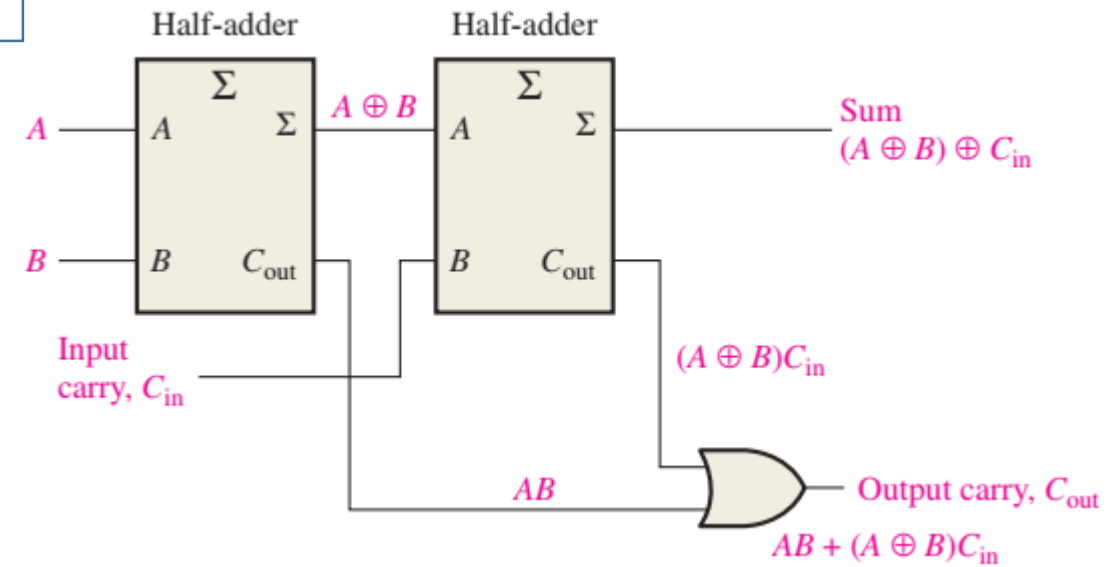
$$\text{Sum} = A \oplus B \oplus C_{in}$$

$$C_{out} = A.B + A.C_{in} + B.C_{in}$$

$$= A.B + (A \oplus B).C_{in}$$



(a) Complete logic circuit for a full-adder



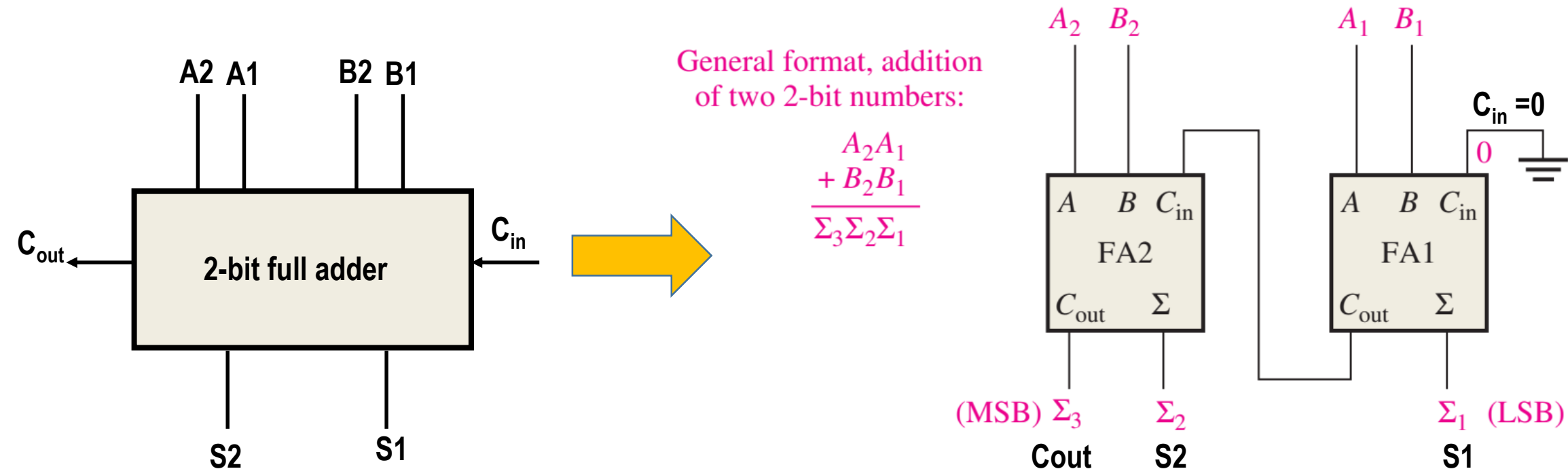
(b) Two half-adders to form a full-adder



- ☐ Half and Full Adders
- ☒ **Parallel Binary Adders**
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer

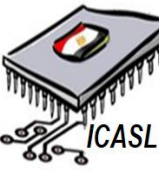
Parallel Binary Adders (1/3)

- ❑ A single **full-adder** is capable of adding **two** 1-bit numbers and an input carry. To add binary numbers with more than one bit, you must use **additional** full-adders.
- ❑ To add two binary numbers, a full-adder (FA) is required for **each bit** in the numbers. So for 2-bit numbers, two adders are needed.
- ❑ The carry output of each adder is connected to the carry input of the **next** higher-order adder.



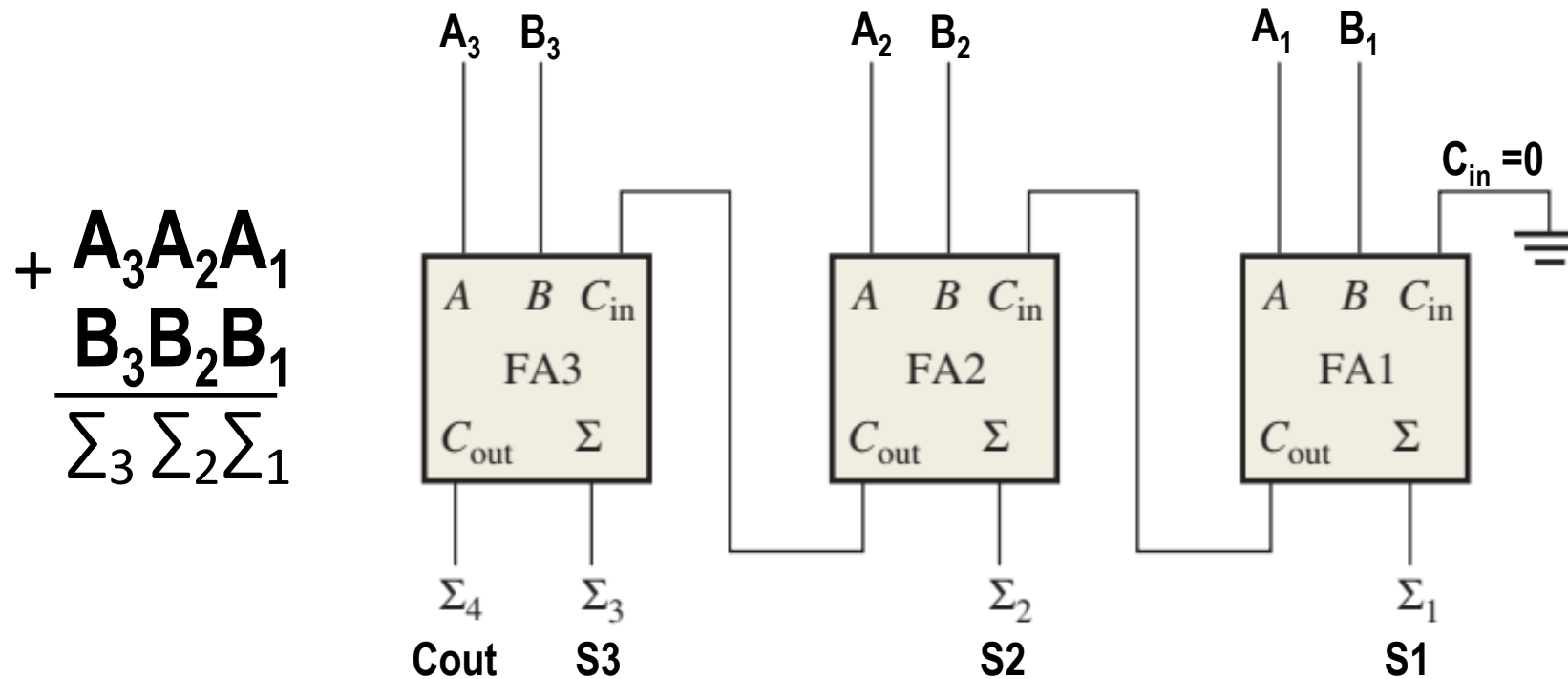
Block diagram of a basic 2-bit parallel adder using two full-adders

Parallel Binary Adders (1/3)



- ❑ A) Design a 3-bit parallel adder
- ❑ B) show the intermediate carries when the binary numbers 101 and 011 are being added

Ans.



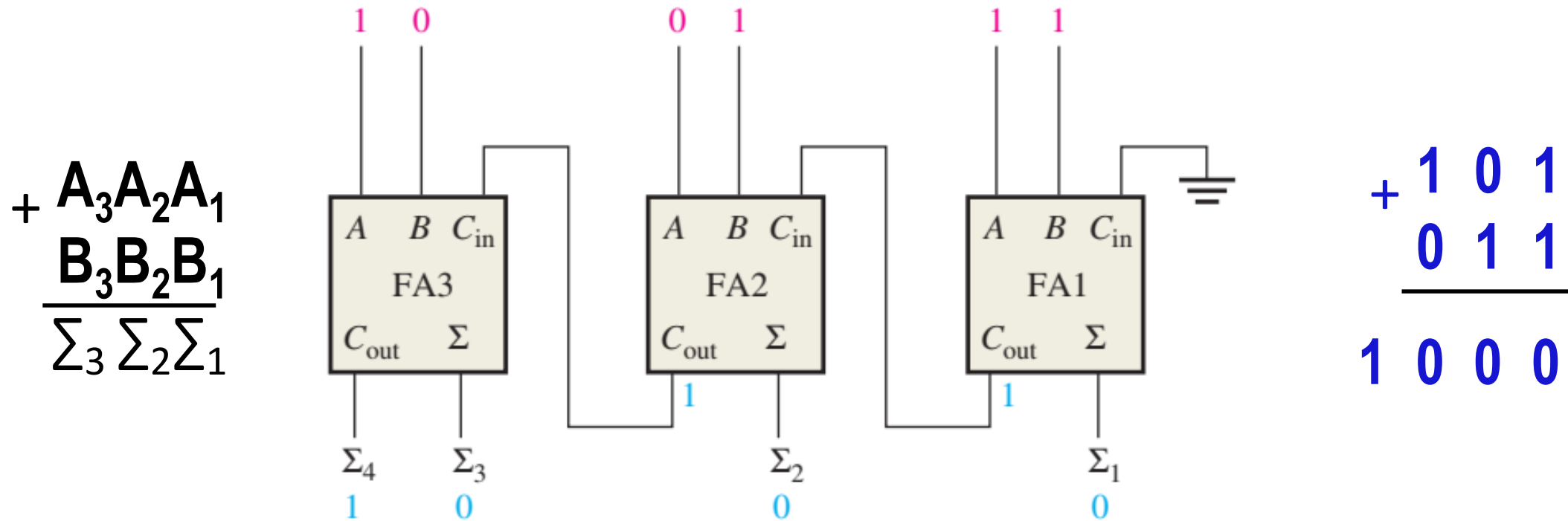
Block diagram of a basic 3-bit parallel adder using two full-adders

Parallel Binary Adders (3/3)



- ☐ A) Design a 3-bit parallel adder
- ☐ B) show the intermediate carries when the binary numbers 101 and 011 are being added

Ans.

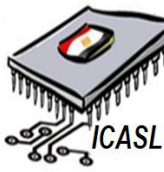


Block diagram of a basic 3-bit parallel adder using two full-adders

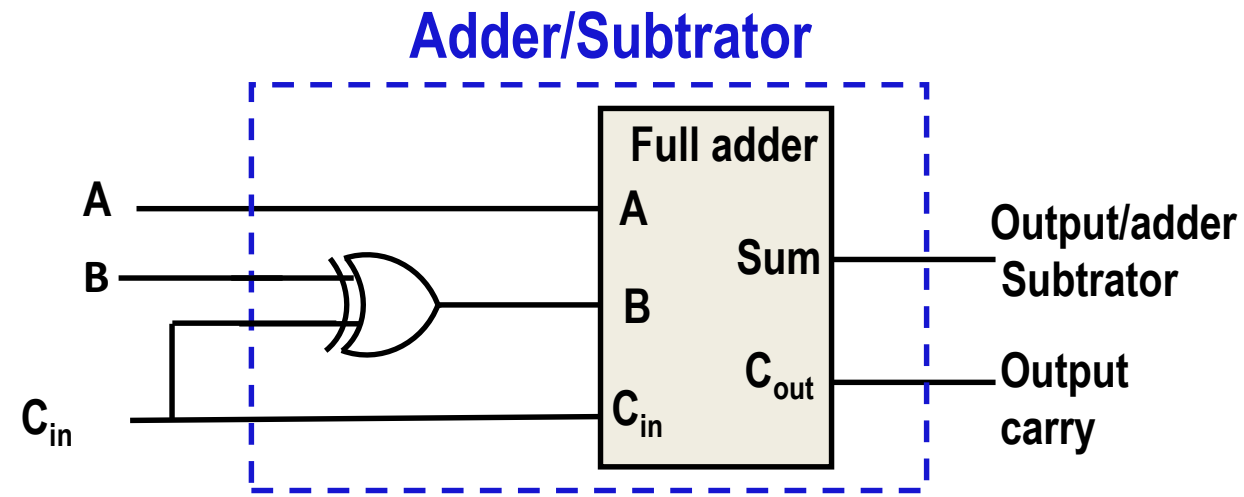
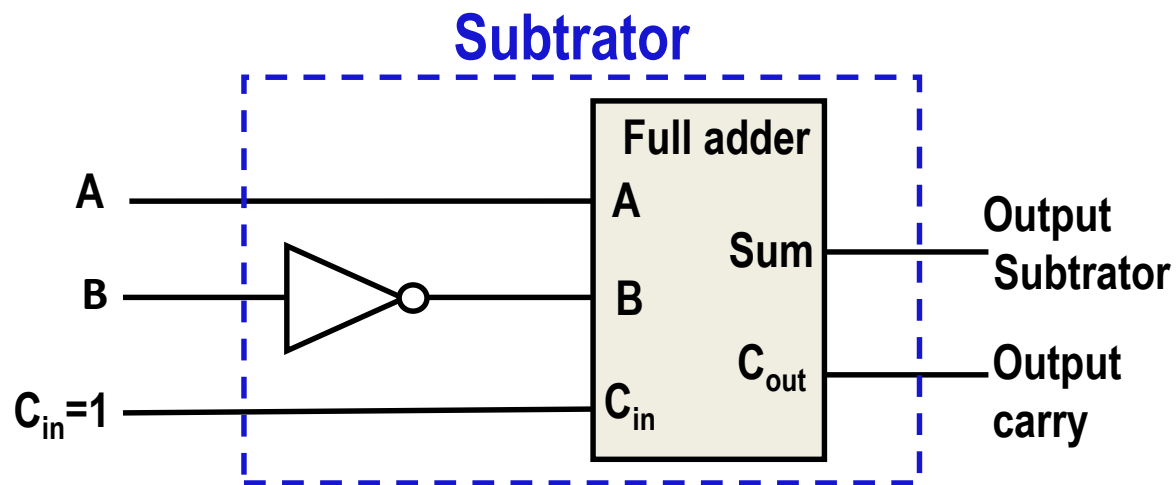


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ **Binary Subtractor**
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer

Binary Subtractor



- ❑ The subtraction of binary numbers can be easily done using 2's complements.
- ❑ The subtraction $A - B$ is done by taking the **2's complement of B** and **adding** it to A .
- ❑ The 2's complement can be obtained by taking the 1's complement and adding 1 to the least significant bit (LSB).



If $C_{in} = 0$, the circuit acts as an Adder

If $C_{in} = 1$, the circuit acts as an Subtractor

*Remember that $B \oplus 0 = B$
and $B \oplus 1 = B'$*

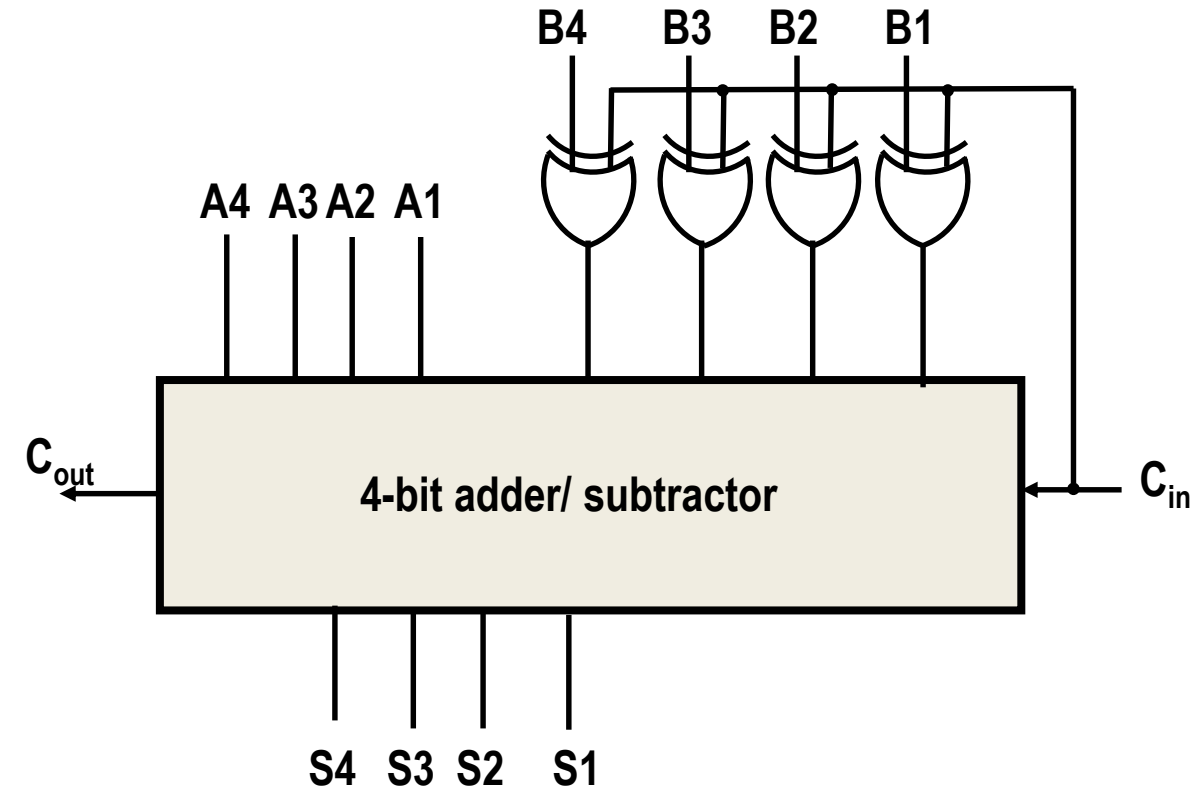
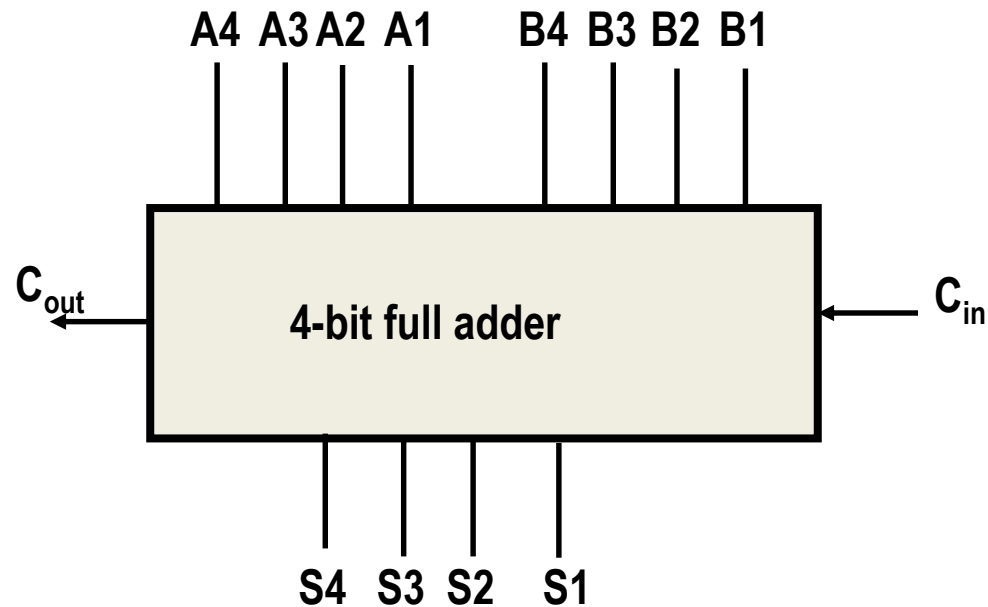


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ **Parallel Binary Adder/Subtractor**
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer

Parallel Binary Adder/Subtractor



- Design a 4-bit parallel adder/Subtractor



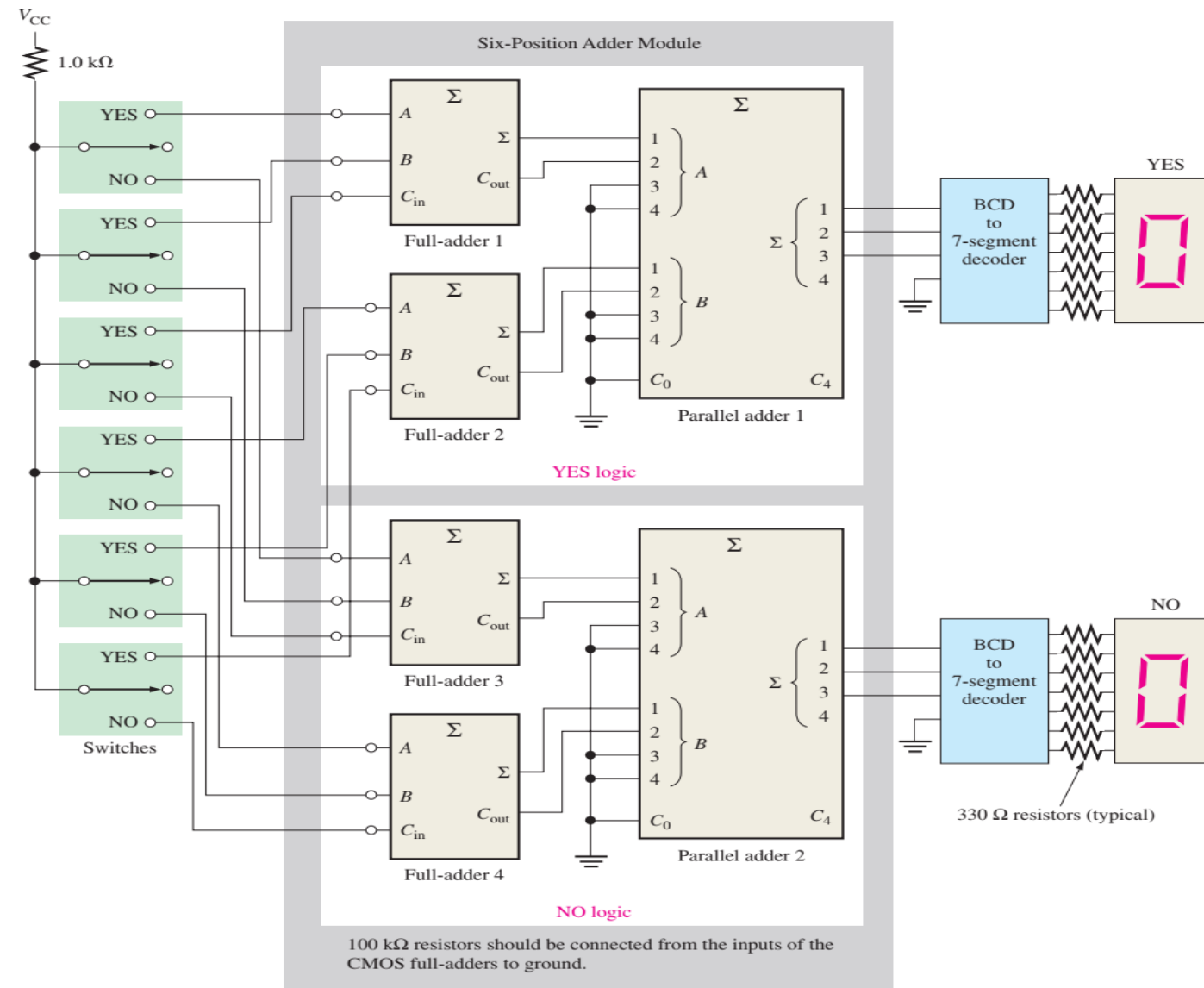
If $C_{in} = 0$, the circuit acts as an Adder $\rightarrow S = A + B$
If $C_{in} = 1$, the circuit acts as a Subtractor $\rightarrow S = A + (-B)$

Application (1/2)



- ❑ An example of **full-adder and parallel adder** application is a simple **voting** system that can be used to simultaneously provide the number of “**yes**” votes and the number of “**no**” votes.
- ❑ This type of system can be used where a group of people are **assembled** and there is a need for immediately determining opinions (for or against), making decisions, or voting on certain issues or other matters.
- ❑ In its **simplest form**, the system includes a switch for “**yes**” or “**no**” selection at each position in the assembly and a digital display for the number of yes votes and one for the number of no votes .
- ❑ The basic system is for a **6-position setup**, but it can be expanded to any number of positions with additional 6-position modules and additional parallel adder and display circuits.

Application (2/2)



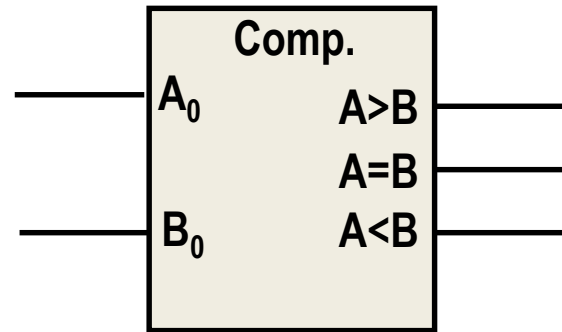
A voting system using full-adders and parallel binary adders



- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ **Comparator**
- ☐ Decoder
- ☐ Encoder
- ☐ Application
- ☐ Multiplexer
- ☐ De-multiplexer

Comparators (1/3)

- ❑ The basic function of a comparator is to compare the magnitudes of **two** binary quantities to determine the relationship of those quantities.
- ❑ Ex.1.Design a comparator circuit to compare **two** numbers {A,B} where each number consists of 1 bit.



Logic symbol

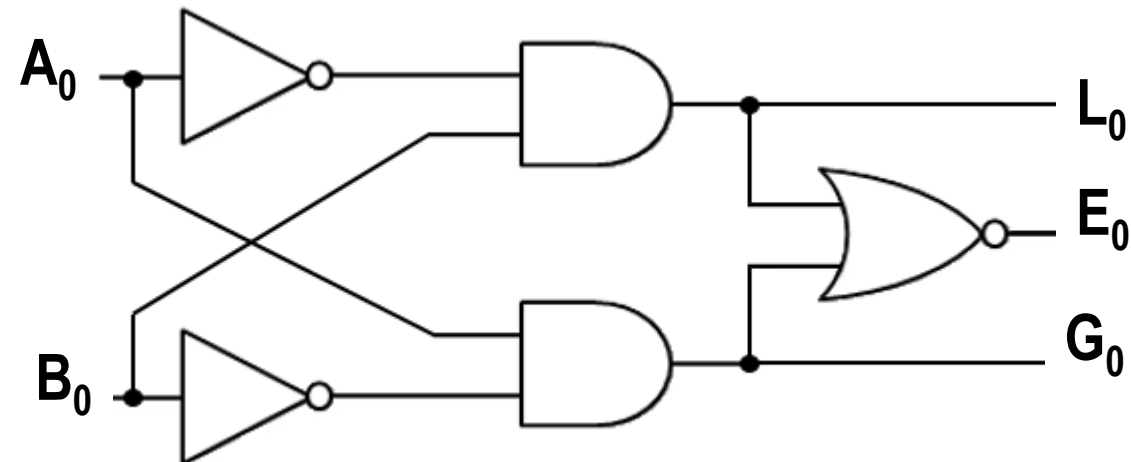
A ₀	B ₀	G ₀	L ₀	E ₀
		A ₀ > B ₀	A ₀ < B ₀	A ₀ = B ₀
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

Ans.

$$G_0 = A_0 \cdot \overline{B_0}$$

$$L_0 = \overline{A_0} \cdot B_0$$

$$E_0 = \overline{A_0} \cdot \overline{B_0} + A_0 \cdot B_0 = \overline{G_0 + L_0}$$

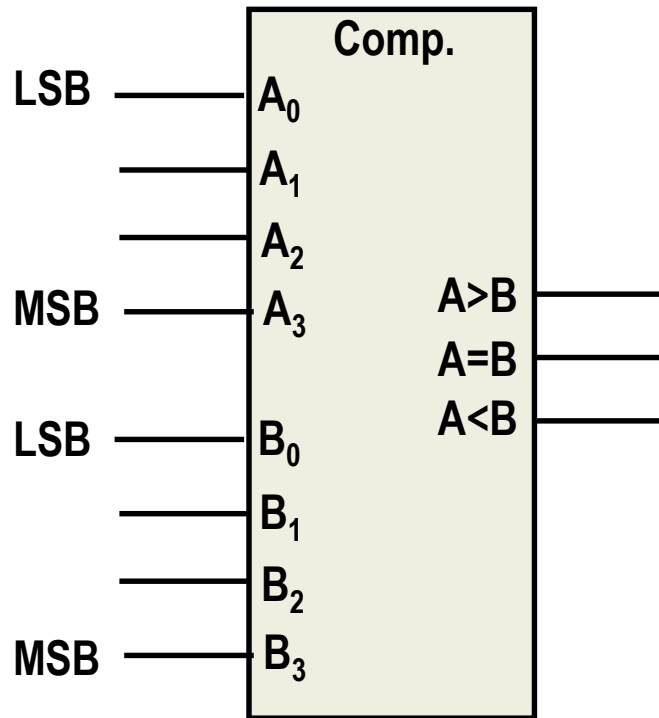


Comparators (2/3)



- Ex2. Design a comparator circuit to compare two numbers {A,B} where each number consists of 4 bit.

Ans.



Logic symbol

Note

$$\begin{matrix} A = & 1 & 0 & 1 & 1 \\ B = & 0 & 1 & 1 & 1 \end{matrix}$$



A > B



$$G = G_3 + E_3 G_2 + E_3 E_2 G_1 + E_3 E_2 E_1 G_0$$

$$\begin{matrix} A = & 1 & 0 & 0 & 1 \\ B = & 1 & 0 & 1 & 0 \end{matrix}$$



A < B



$$L = L_3 + E_3 L_2 + E_3 E_2 L_1 + E_3 E_2 E_1 L_0$$

$$\begin{matrix} A = & 0 & 1 & 0 & 1 \\ B = & 0 & 1 & 0 & 1 \end{matrix}$$



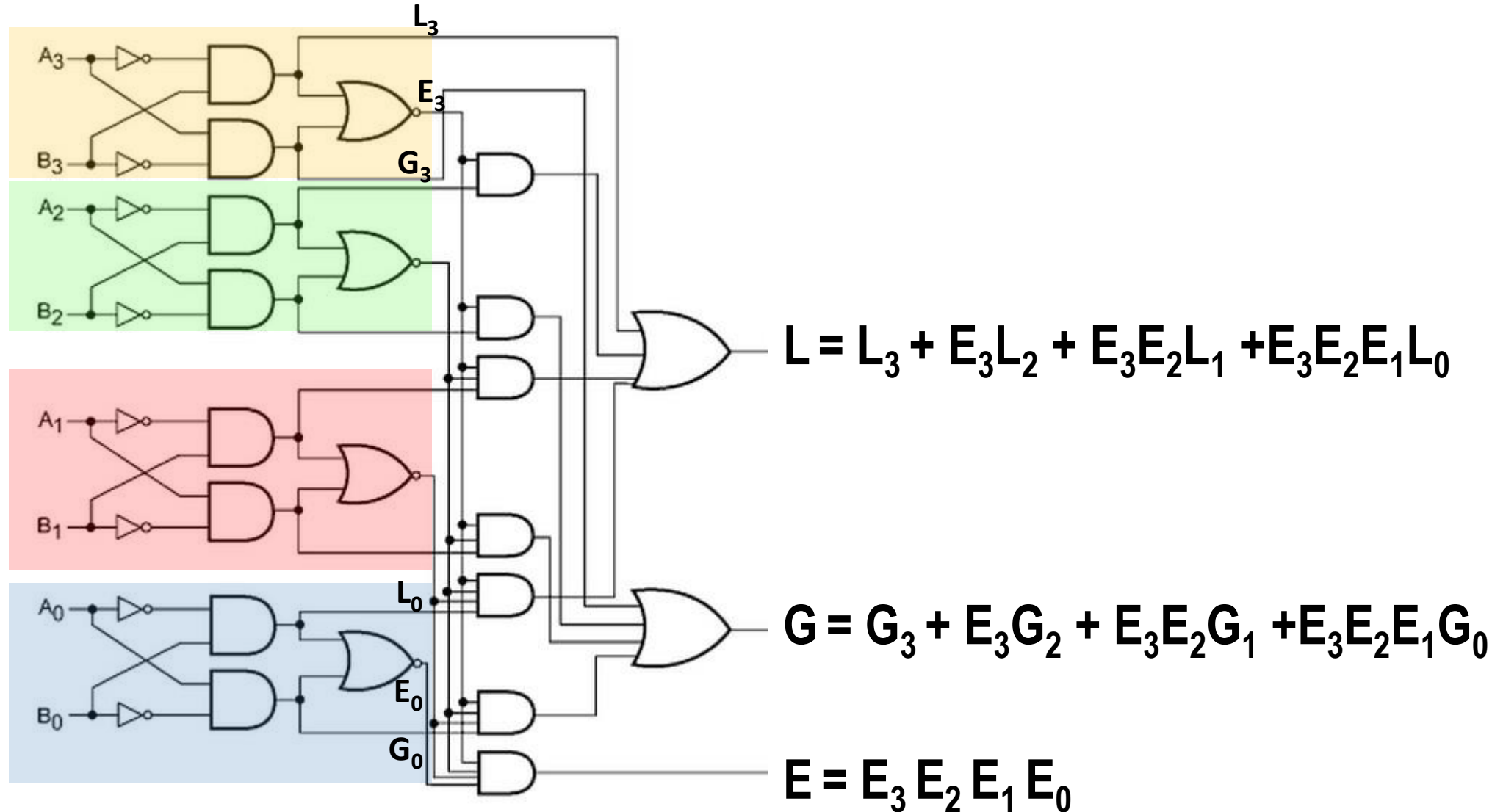
A = B



$$E = E_3 E_2 E_1 E_0$$

Those two numbers would be equal if all pairs of their significant digits are equal: $A_3=B_3$, $A_2=B_2$, $A_1=B_1$, $A_0=B_0$

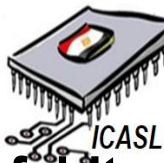
Comparators (3/3)



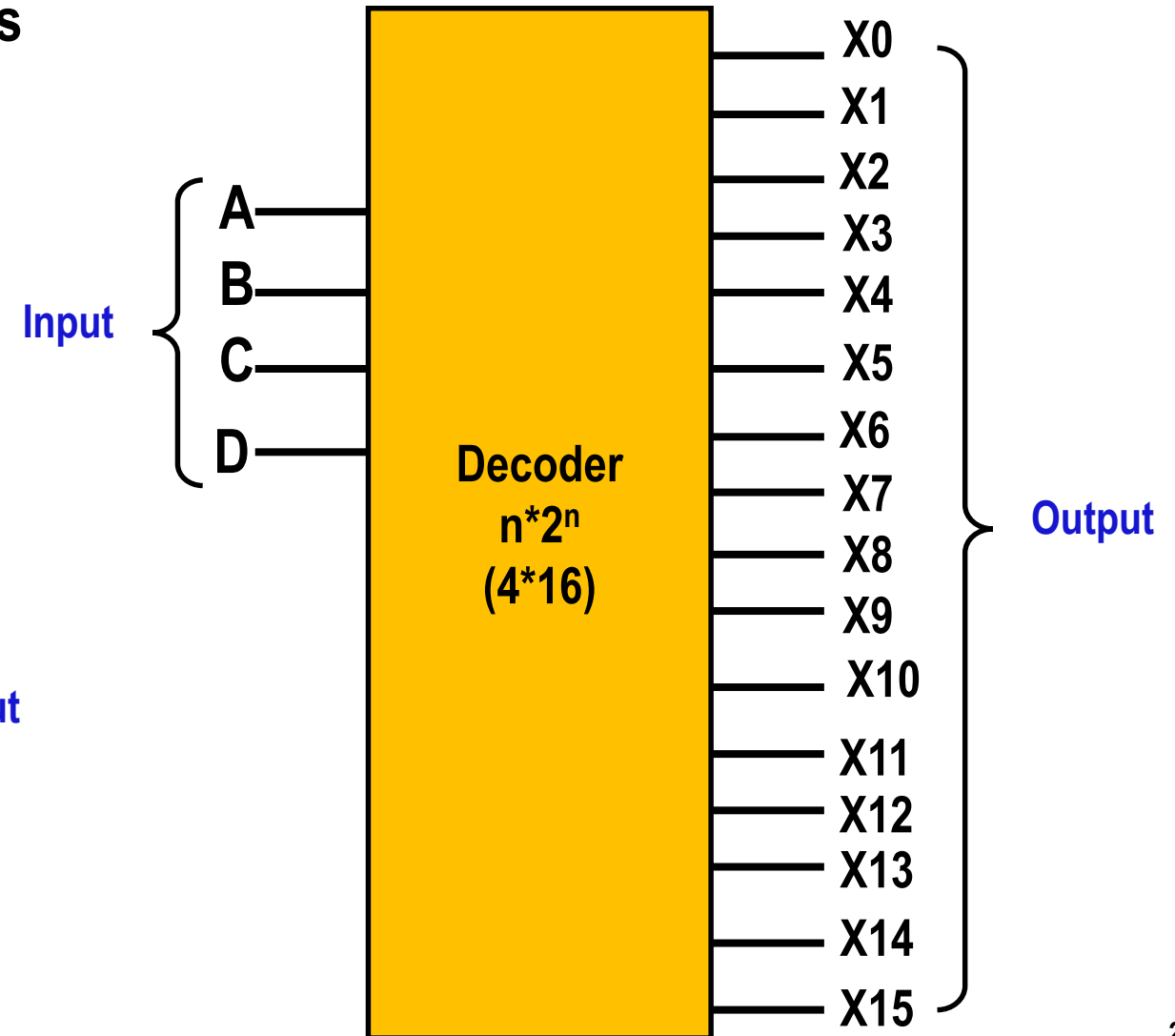
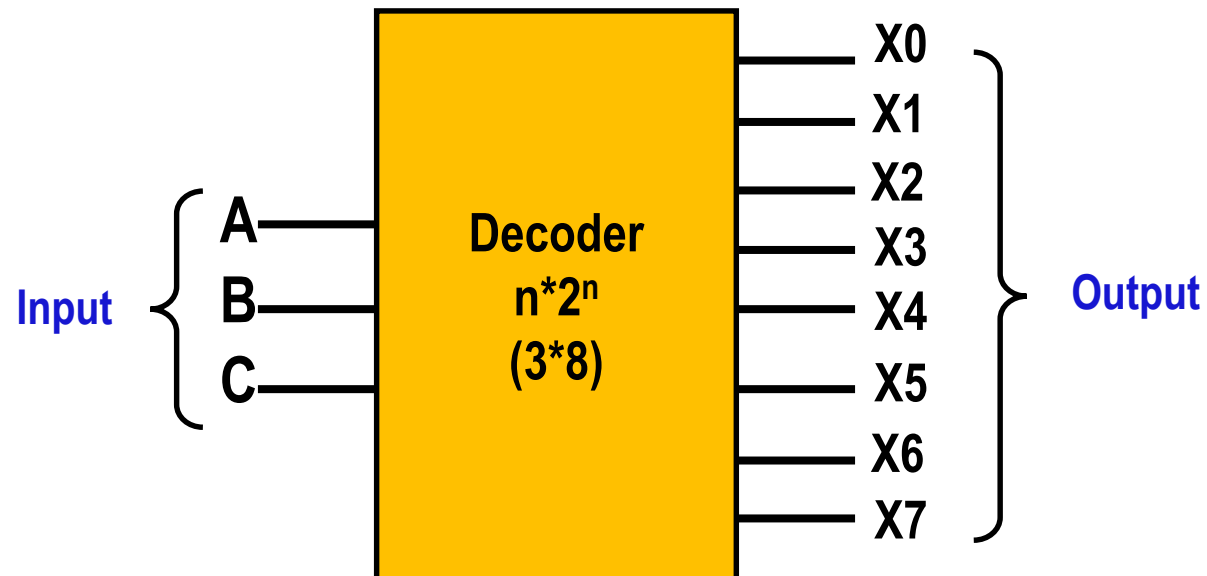
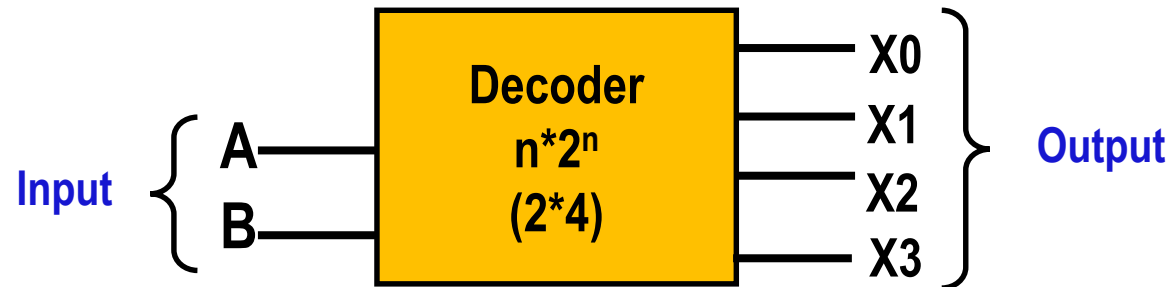


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ **Decoder**
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer

Decoder (1/5)

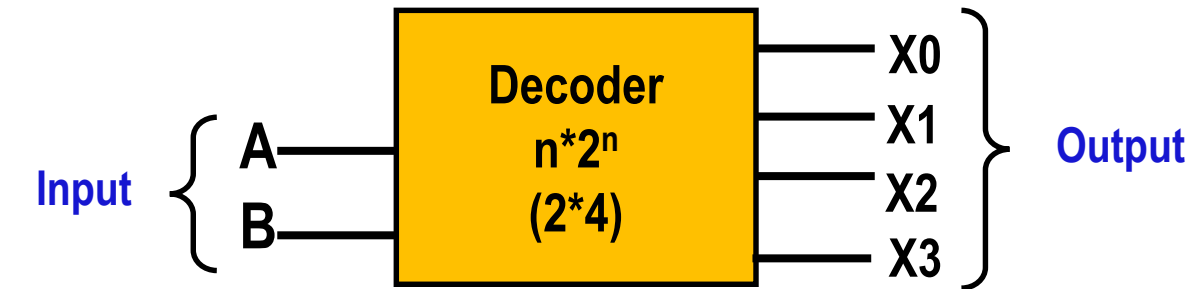


- ❑ A decoder is a digital circuit that **detects** the presence of a specified combination of bits (code) on its inputs and **indicates the presence of that code** by a specified output level .
- ❑ A decoder has **n** input lines form **2^n** output lines



Decoder (2/5)

□ Ex.1 Design a 2-bit decoder with active-high outputs



Input		Output			
A	B	X0	X1	X2	X3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

A	B	
	0	1
0	1	0
1	0	0

$X0 = \overline{A} \cdot \overline{B}$

A	B	
	0	1
0	0	1
1	0	0

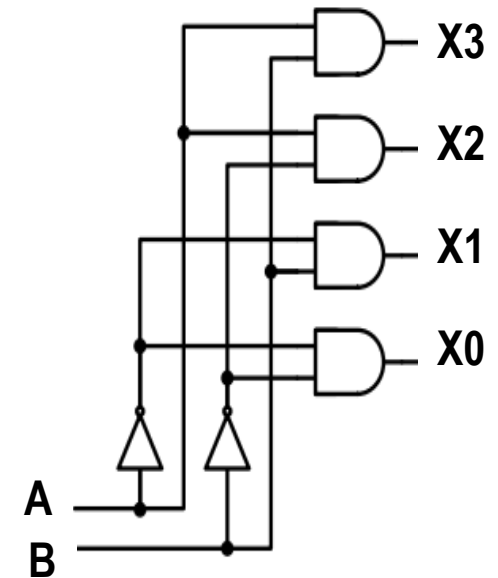
$X1 = \overline{A} \cdot B$

A	B	
	0	1
0	0	0
1	1	0

$X2 = A \cdot \overline{B}$

A	B	
	0	1
0	0	0
1	0	1

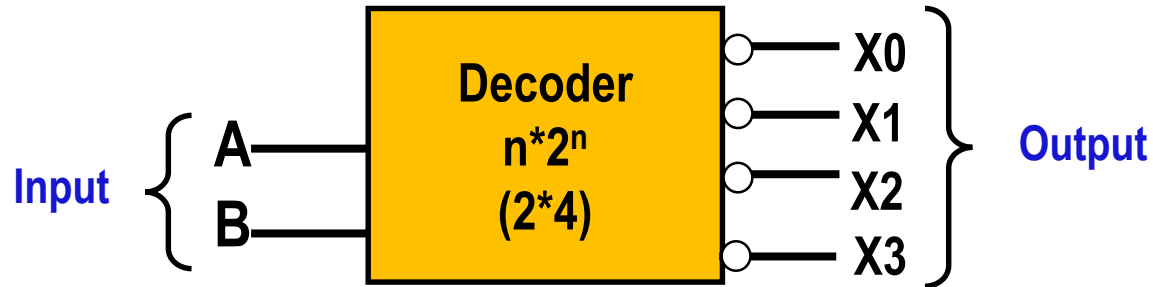
$X3 = A \cdot B$



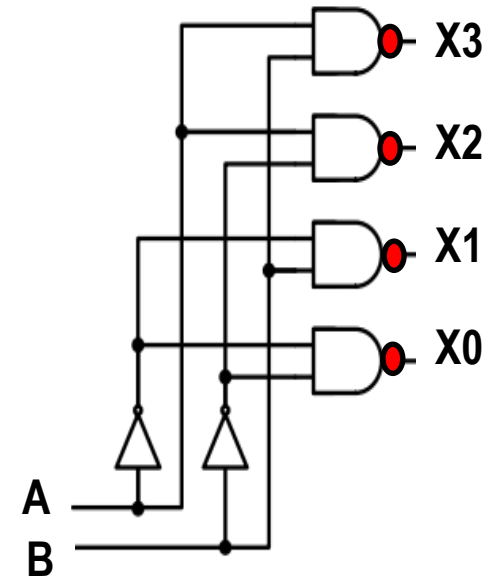
Decoder (3/5)



□ Ex.2 Design a 2-bit decoder with active-Low outputs



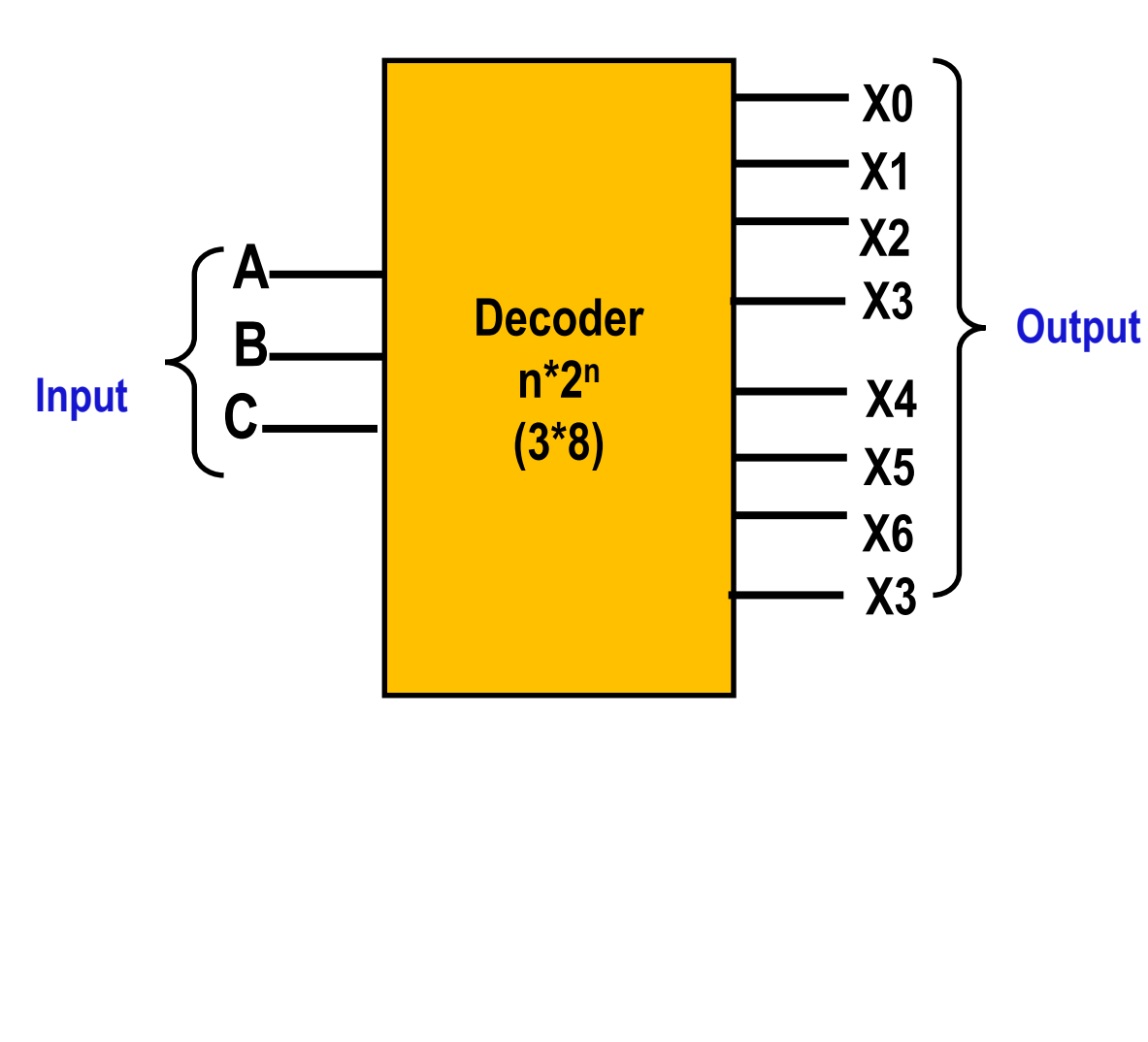
Input		Output			
A	B	X0	X1	X2	X3
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0



Decoder (4/5)



□ Ex.3 Design a 3-bit decoder with active-high outputs



Input			Output							
A	B	C	X0	X1	X2	X3	X4	X5	X6	X7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

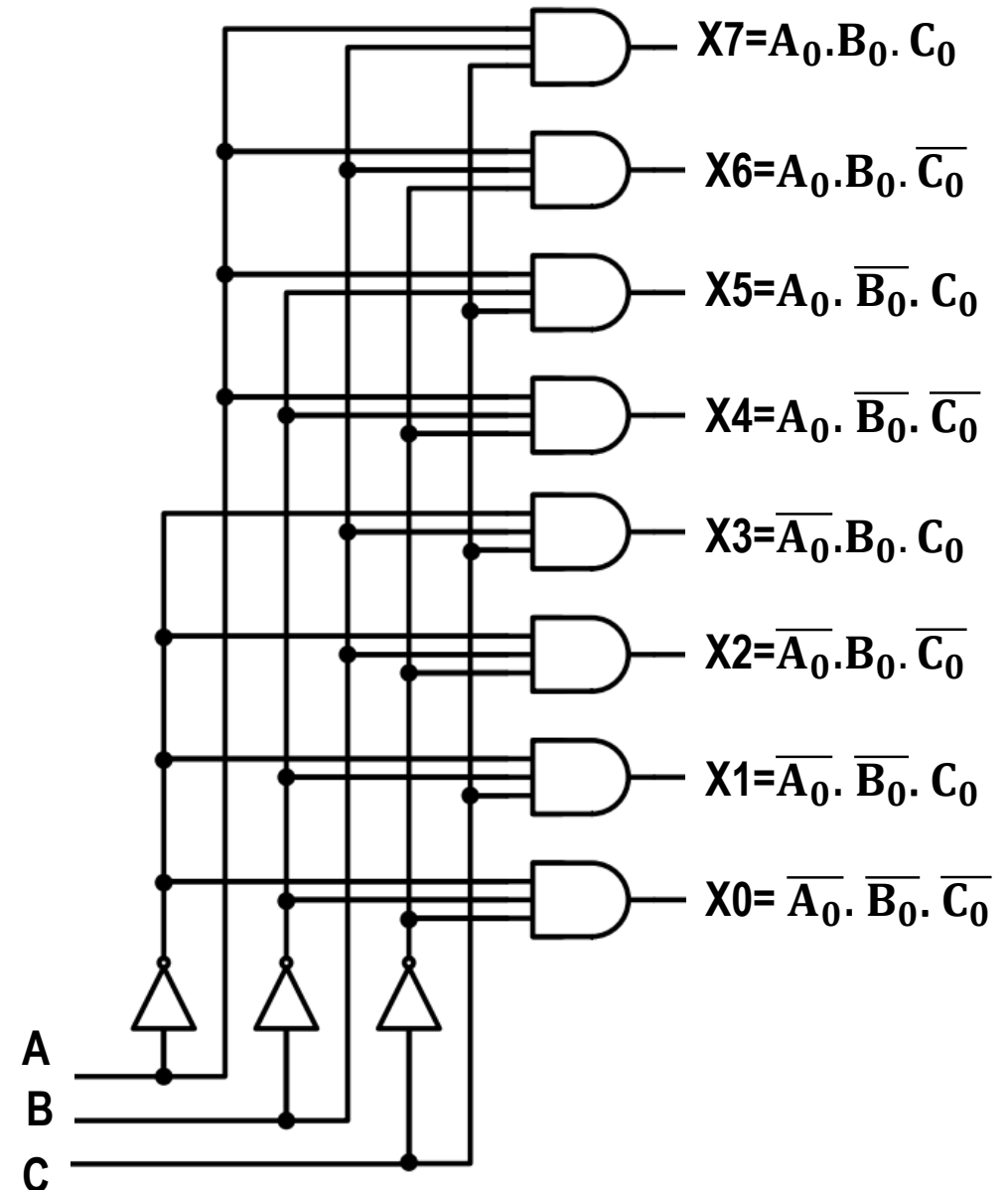
Decoder (5/5)



❑ Ex.3 Design a 3-bit decoder with active-high outputs

Input			Output							
A	B	C	X0	X1	X2	X3	X4	X5	X6	X7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

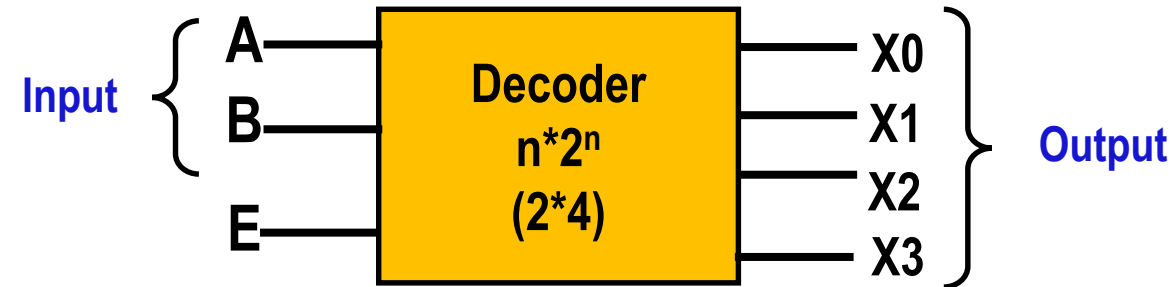
$$\begin{array}{lll} X0 = \overline{A_0} \cdot \overline{B_0} \cdot \overline{C_0} & X2 = \overline{A_0} \cdot B_0 \cdot \overline{C_0} & X4 = A_0 \cdot \overline{B_0} \cdot \overline{C_0} \\ X1 = \overline{A_0} \cdot \overline{B_0} \cdot C_0 & X3 = \overline{A_0} \cdot B_0 \cdot C_0 & X5 = A_0 \cdot \overline{B_0} \cdot C_0 \\ X6 = A_0 \cdot B_0 \cdot \overline{C_0} & X7 = A_0 \cdot B_0 \cdot C_0 & \end{array}$$



Decoder Expander (1/2)



□ Ex.1 Design a 2-bit decoder with enable line and with active-high outputs



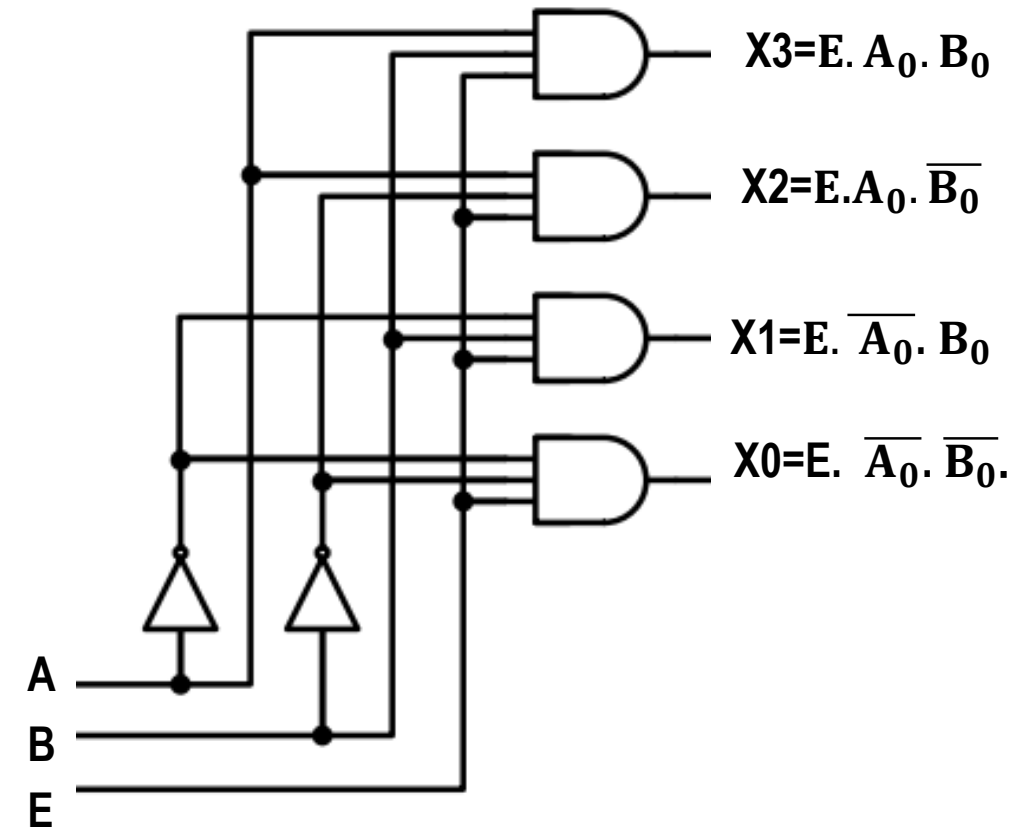
Input			Output			
E	A	B	X0	X1	X2	X3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	X	X	0	0	0	0

$$X0 = E \cdot \overline{A_0} \cdot \overline{B_0}$$

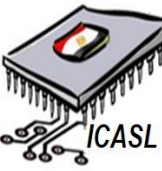
$$X1 = E \cdot \overline{A_0} \cdot B_0$$

$$X2 = E \cdot A_0 \cdot \overline{B_0}$$

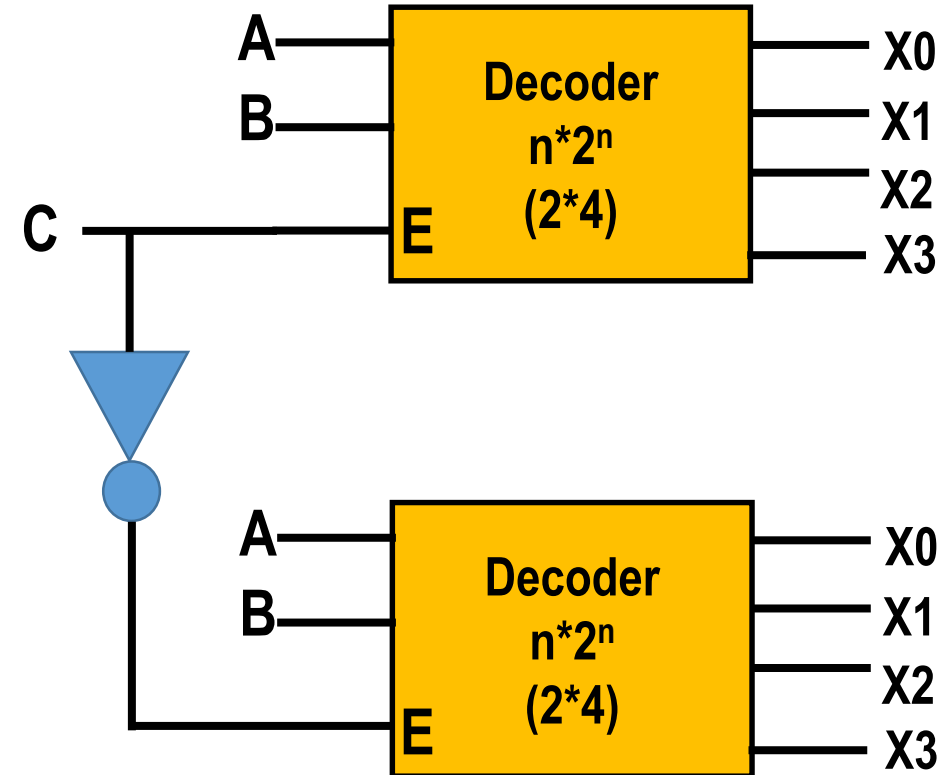
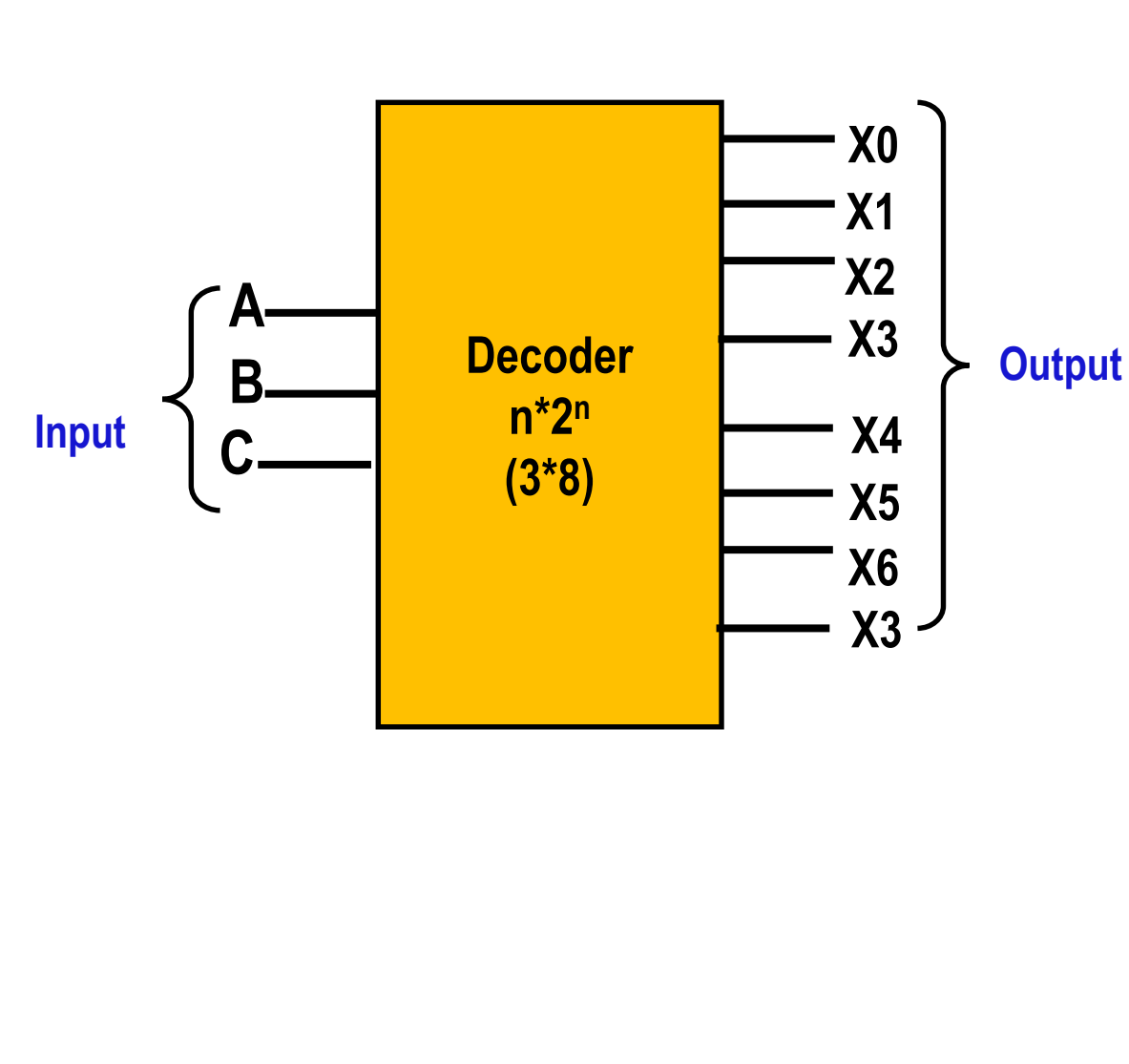
$$X3 = E \cdot A_0 \cdot B_0$$



Decoder Expander (2/2)



- Design a 3-bit decoder with 2-bit decoders whose an enable line.





- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☒ Encoder
- ☐ Comparator
- ☐ Multiplexer
- ☐ De-multiplexer

Encoder (1/2)



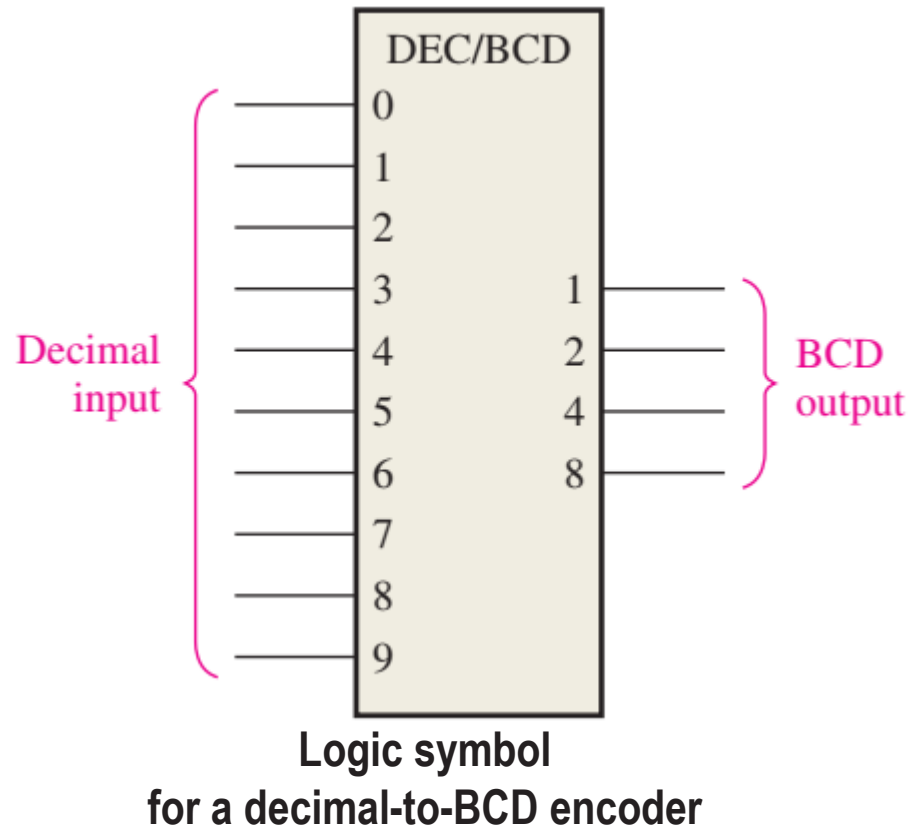
- ❑ An encoder is a combinational logic circuit that essentially performs a “**reverse**” decoder function.
- ❑ An encoder accepts an active level on **one** of its inputs representing a digit, such as a **decimal** and converts it to a coded output, such as **BCD or binary**.
- ❑ Encoders can also be devised to encode various symbols and alphabetic characters.
- ❑ The process of converting from **familiar** symbols or numbers to a coded format is called *encoding*

❑ Ex. The Decimal-to-BCD Encoder

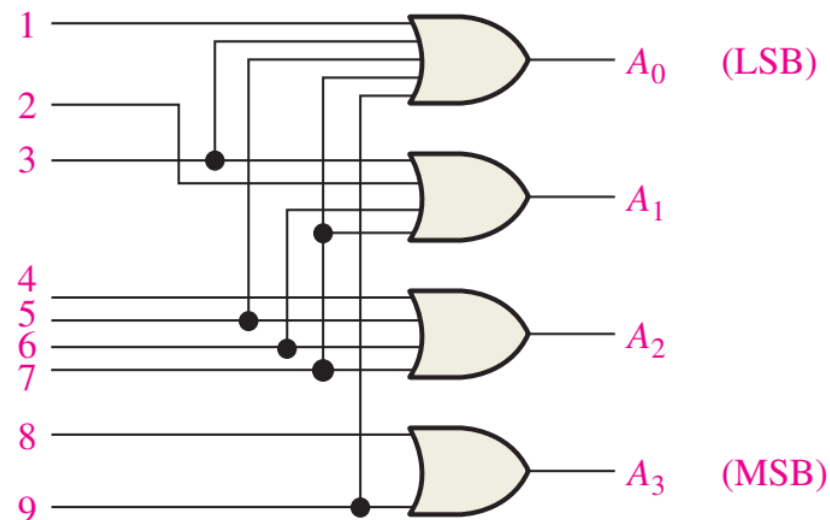
Encoder (2/2)



- This type of encoder has **ten** inputs—one for each decimal digit—and **four** outputs corresponding to the BCD code



Decimal Digit	BCD Code			
	A_3	A_2	A_1	A_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1



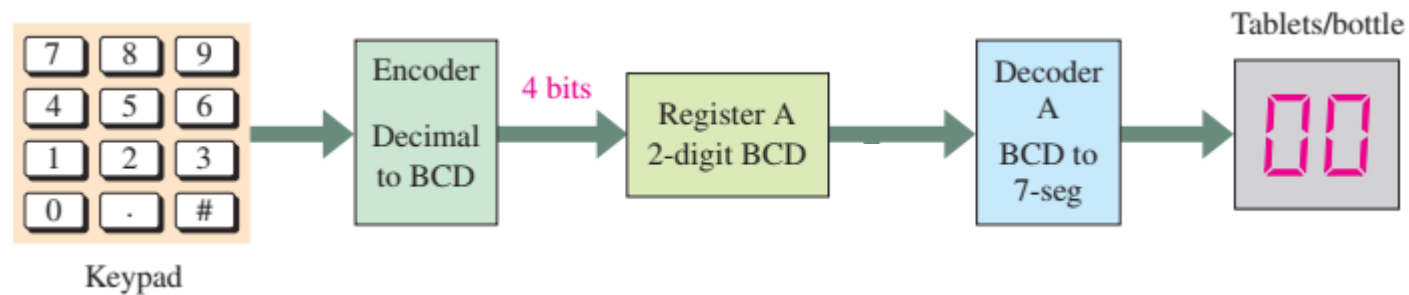
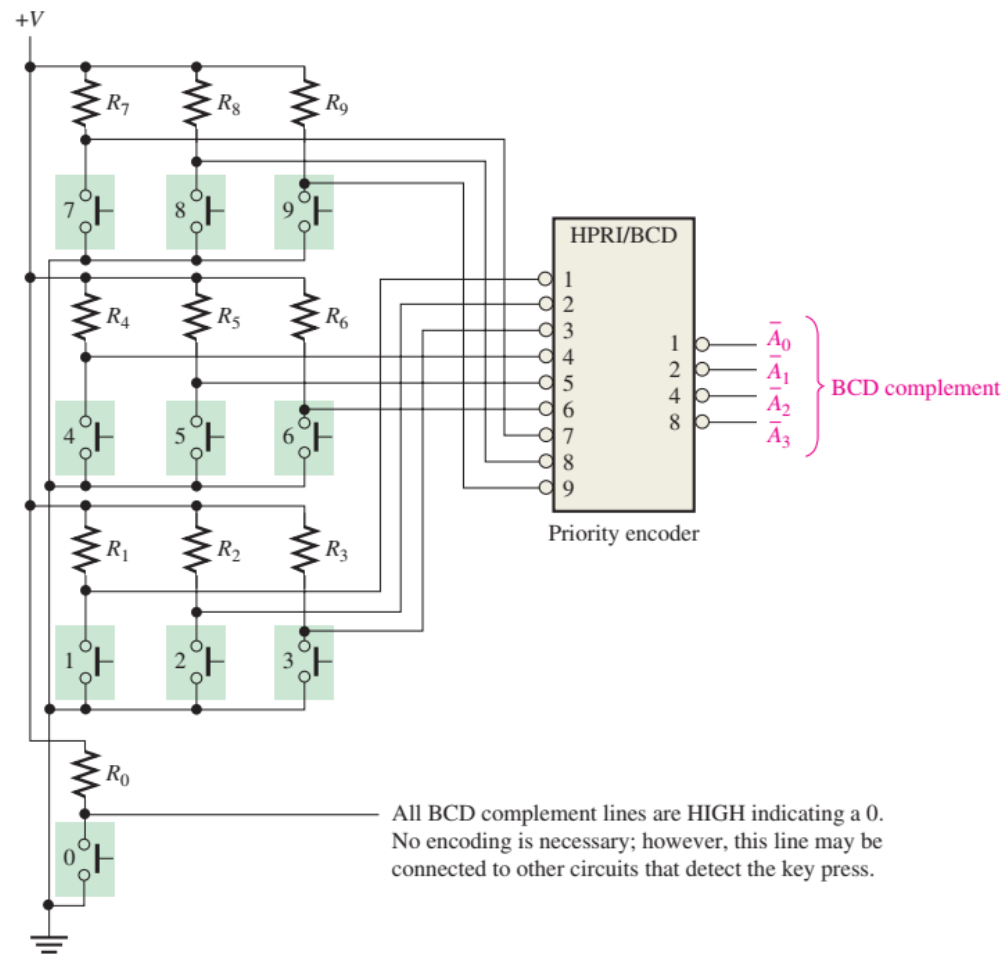
Basic logic diagram of a decimal-to-BCD encoder



❑ A 7-Segment Display Multiplexer

- ❑ The **ten** decimal digits on a numeric **keypad** must be encoded for processing by the logic circuitry.
- ❑ In this example, when one of the keys is **pressed**, the decimal digit is encoded to the corresponding BCD code.
- ❑ A simple **keyboard** encoder arrangement using a priority encoder can be designed .
- ❑ The keys are represented by **ten** push-button switches, each with a pull-up resistor to +5V.
- ❑ The pull-up resistor ensures that the line is **HIGH** when a key is not depressed.
- ❑ When a key is depressed, the line is connected to ground, and a **LOW** is applied to the corresponding encoder input.
- ❑ The zero key is not connected because the BCD output represents zero when none of the other keys is depressed.

Application



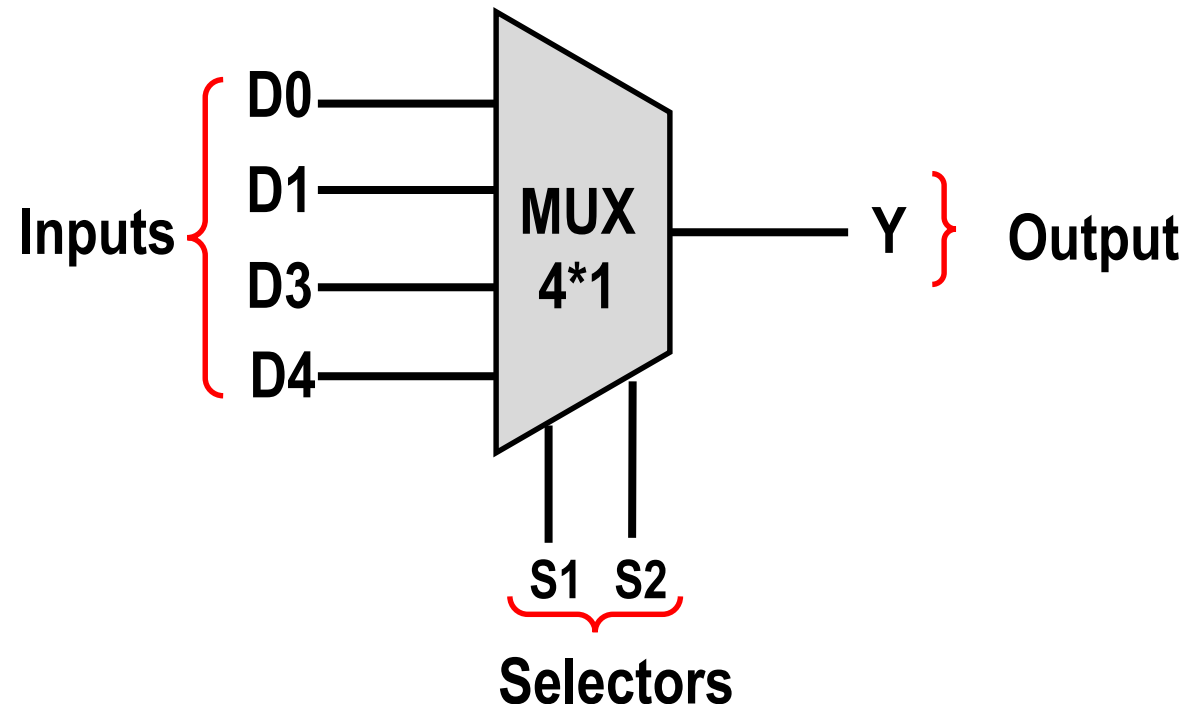
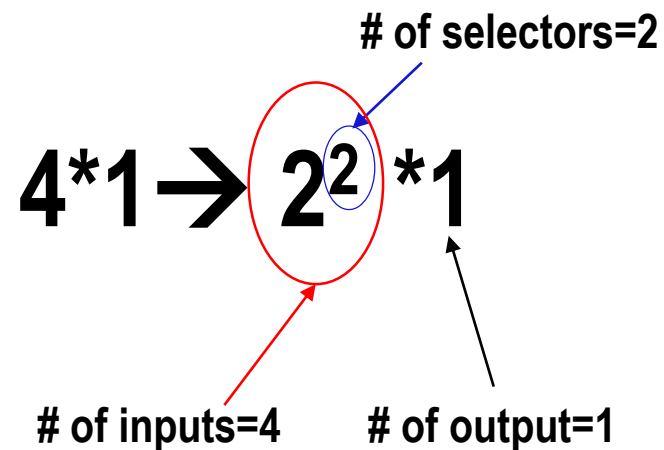


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ **Multiplexer**
- ☐ De-multiplexer

Multiplexers (1/2)



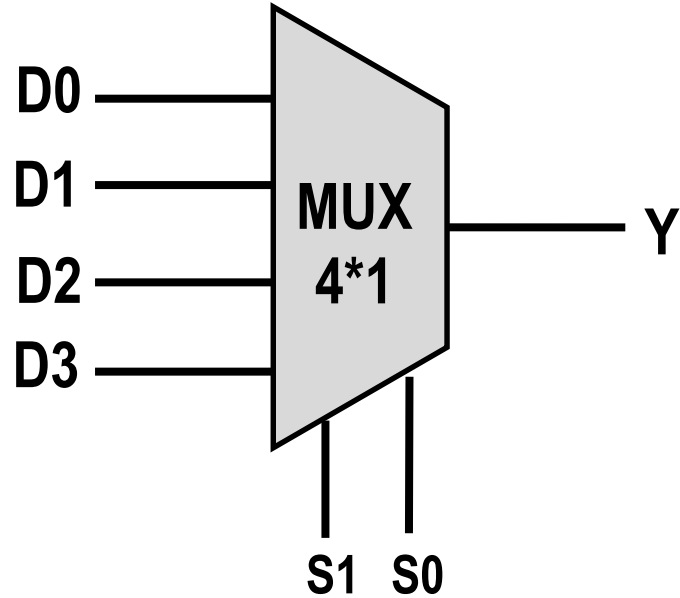
- ❑ A multiplexer (MUX) is a device that allows digital information from several sources to be routed onto a single line for transmission over that line to a common destination.
- ❑ The basic multiplexer has several data-input lines and a single output line.
- ❑ It also has data-select inputs, which permit digital data on any one of the inputs to be switched to the output line.
- ❑ Multiplexers are also known as data selectors.



Multiplexers (2/2)

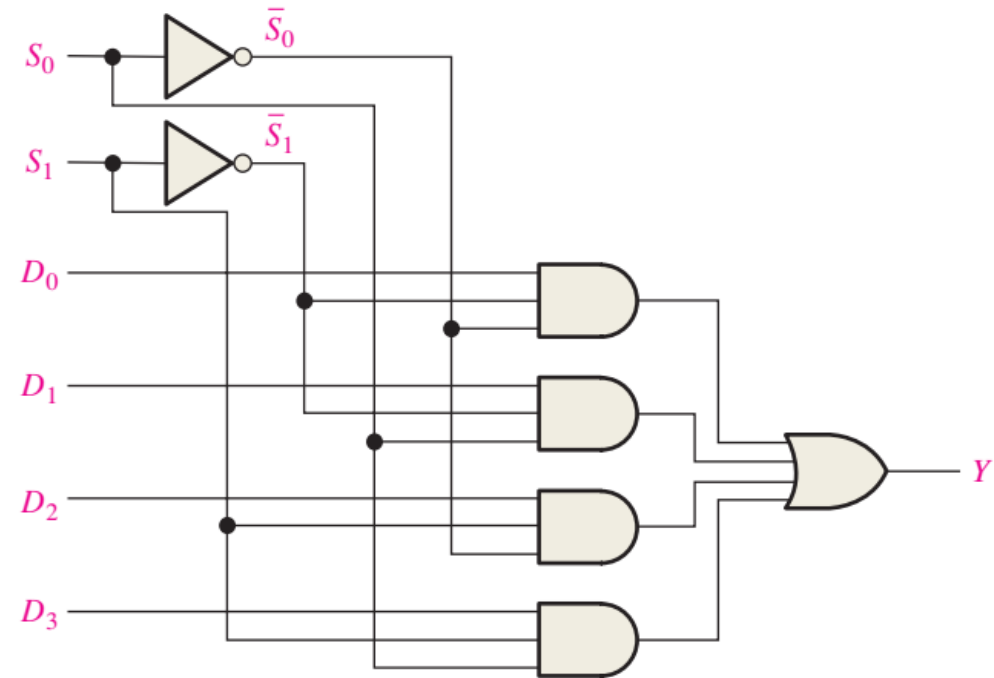


□ Ex1. Design a 4*1 multiplexer using logic gates



Data-Select Inputs		Input Selected
S_1	S_0	
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

$$Y = D_0\bar{S}_1\bar{S}_0 + D_1\bar{S}_1S_0 + D_2S_1\bar{S}_0 + D_3S_1S_0$$

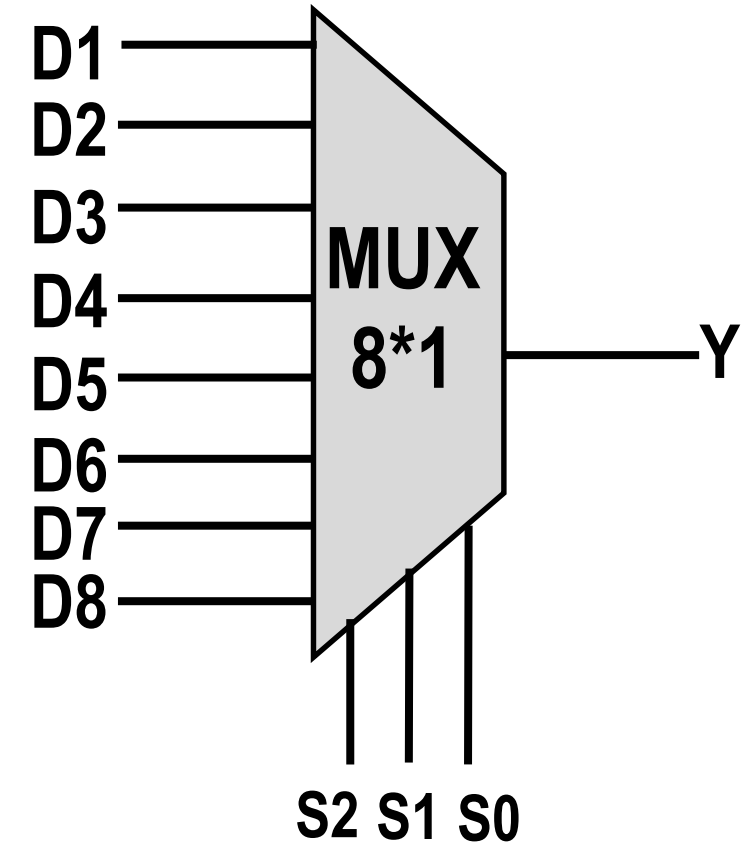
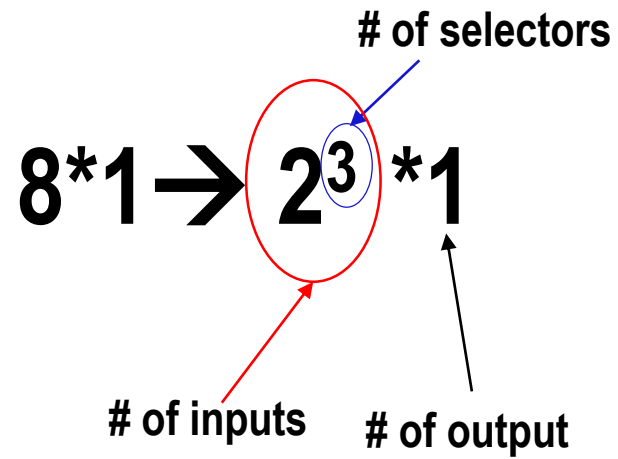


Logic diagram for a 4*1-input multiplexer

Multiplexers (1/2)



□ Ex1. Design a 8*1 multiplexer using logic gates



Multiplexers (1/2)

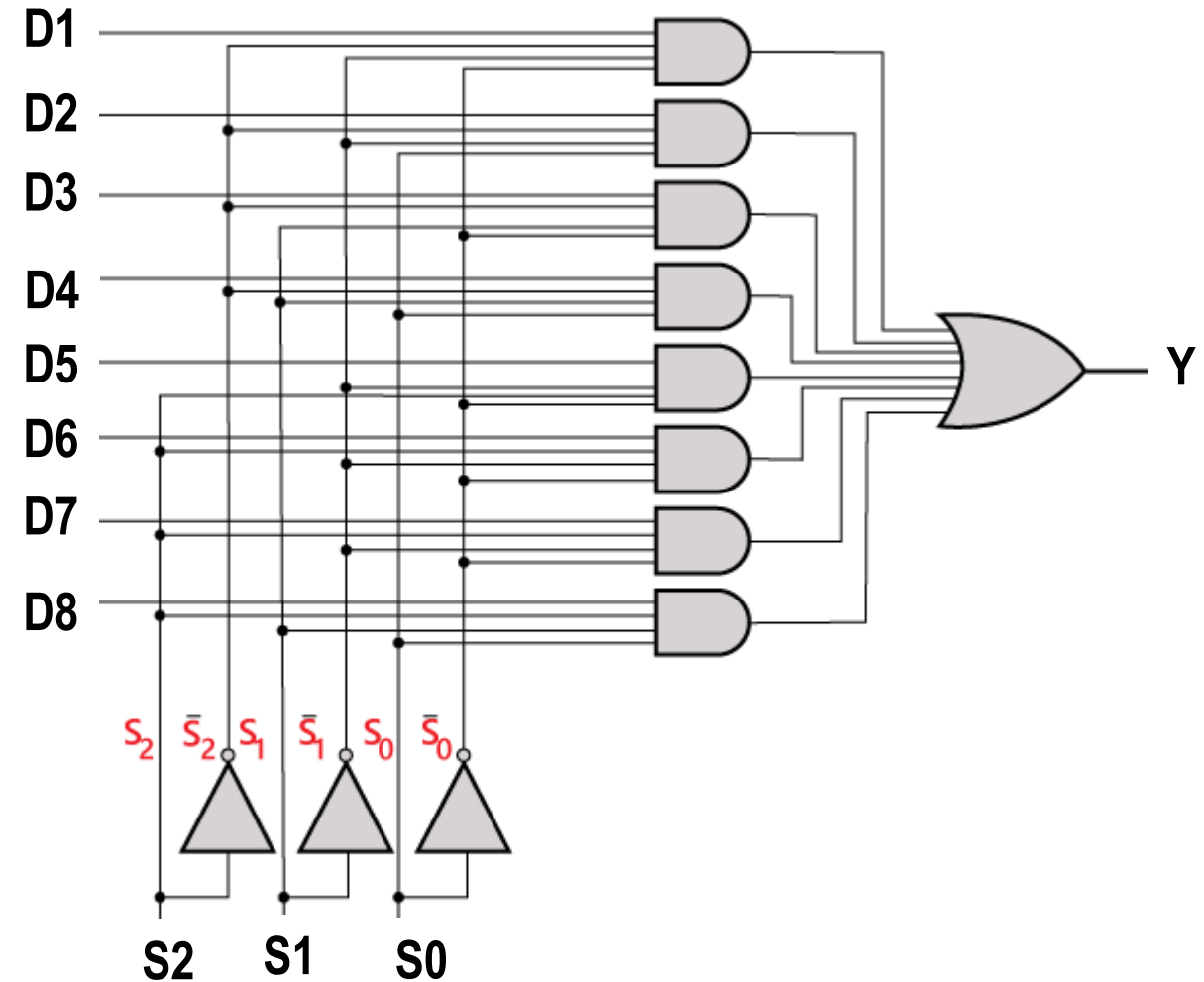


□ Ex1. Design a 8*1 multiplexer using logic gates

Truth table

S2	S1	S0	F
0	0	0	D1
0	0	1	D2
0	1	0	D3
0	1	1	D4
1	0	0	D5
1	0	1	D6
1	1	0	D7
1	1	1	D8

$$Y = D_1 \cdot \overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0} + D_2 \cdot \overline{S_2} \cdot \overline{S_1} \cdot S_0 + D_3 \cdot \overline{S_2} \cdot S_1 \cdot \overline{S_0} + D_4 \cdot \overline{S_2} \cdot S_1 \cdot S_0 + D_5 \cdot S_2 \cdot \overline{S_1} \cdot \overline{S_0} + D_6 \cdot S_2 \cdot \overline{S_1} \cdot S_0 + D_7 \cdot S_2 \cdot S_1 \cdot \overline{S_0} + D_8 \cdot S_2 \cdot S_1 \cdot S_0$$



Logic diagram for a 8*1-input multiplexer



- Design a 3*1 multiplexer using logic gates

Assignment

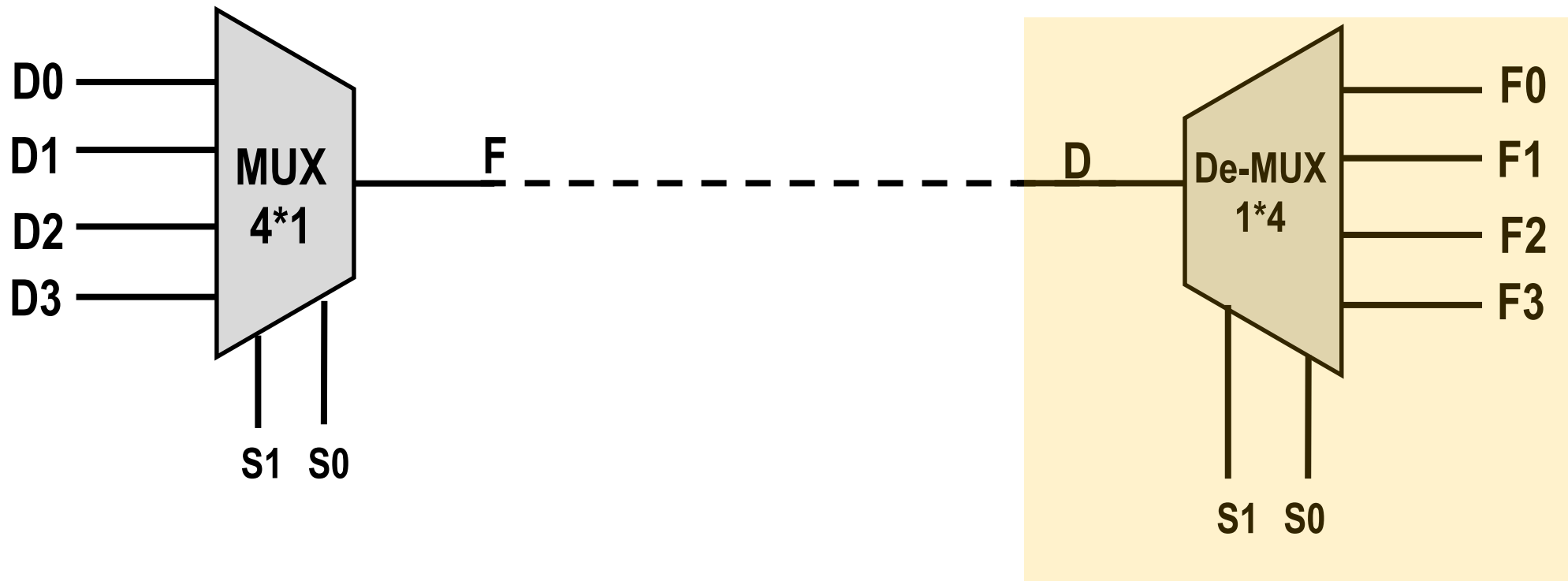


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer

Demultiplex (1/2)



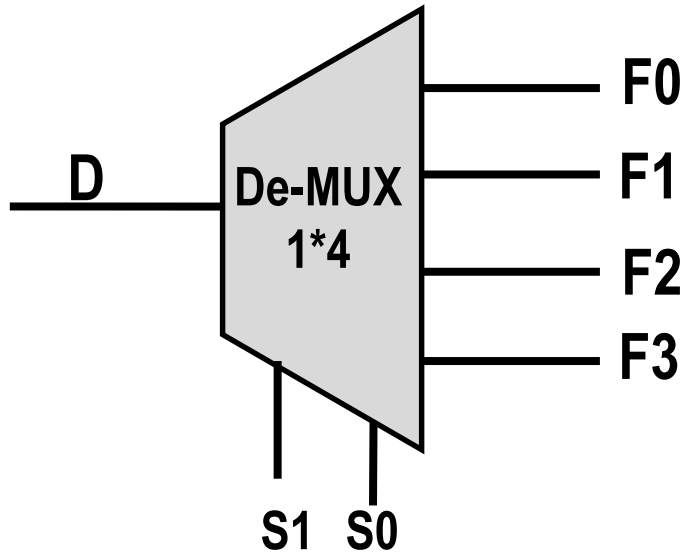
- ❑ A demultiplexer (**DEMUX**) basically reverses the multiplexing function.
- ❑ It takes digital information from one line and distributes it to a given number of output lines.
- ❑ For this reason, the **demultiplexer** is also known as a data distributor.



Demultiplex (2/2)



□ Ex1. Design a 1*4 multiplexer using logic gates



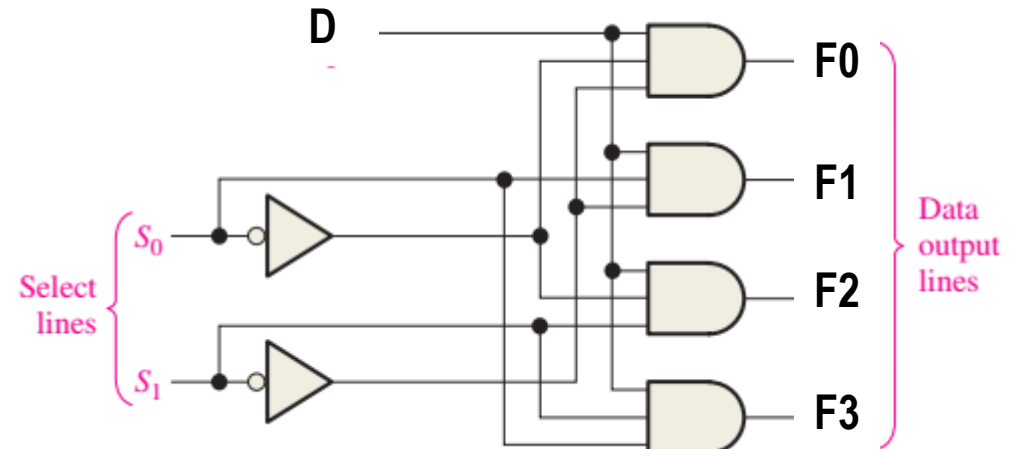
Selectors		Output			
S1	S0	F0	F1	F2	F3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

$$F0 = D \cdot \overline{S_1} \cdot \overline{S_0}$$

$$F1 = D \cdot \overline{S_1} \cdot S_0$$

$$F2 = D \cdot S_1 \cdot \overline{S_0}$$

$$F3 = D \cdot S_1 \cdot S_0$$



A 1-line-to-4-line demultiplexer



Parity Generators/Checkers (1/2)



- ❑ Errors can occur as digital codes are being transferred from one point to another within a digital system.
- ❑ The errors take the form of undesired changes in the bits that make up the coded information; that is, a 1 can change to a 0, or a 0 to a 1, **because of component malfunctions or electrical noise.**
- ❑ In most digital systems, the **probability** that even a single bit error will occur is **very small**, and the likelihood that more than one will occur is even smaller.
- ❑ Nevertheless, when an error occurs undetected, it can cause serious problems in a digital system.

A parity bit indicates if the number of 1s in a code is even or odd for the purpose of error detection.

The sum of an even number of 1s is always 0, and the sum of an odd number of 1s is always 1.

Parity Generators/Checkers



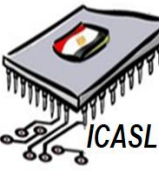
□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

(b) Using Logic gate

Ans.

Parity Generators/Checkers



□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

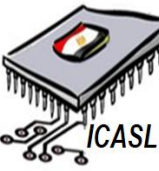
(a) Using universal logic gates

(b) Using Logic gate

Ans.



Implementation using Universal Logic Gates



❑ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

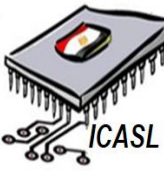
(b) Using Logic gates

Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



Implementation using Universal Logic Gates



❑ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

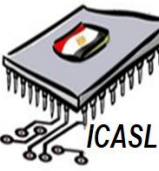
(b) Using Logic gate

Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



Implementation using Universal Logic Gates



❑ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

(b) Using Logic gate

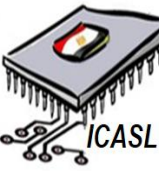
Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



		B			
		00	01	11	10
A	BC	0	1	0	1
	1	1	0	1	0

Implementation using Universal Logic Gates



❑ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

(b) Using Logic gate

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Ans.



		B			
		00	01	11	10
A	0	0	1	0	1
	1	1	0	1	0

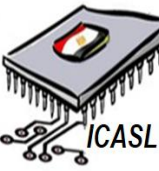
BC

C

The output function is obtained by using Sum Of Product (SOP)

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$

Implementation using Universal Logic Gates



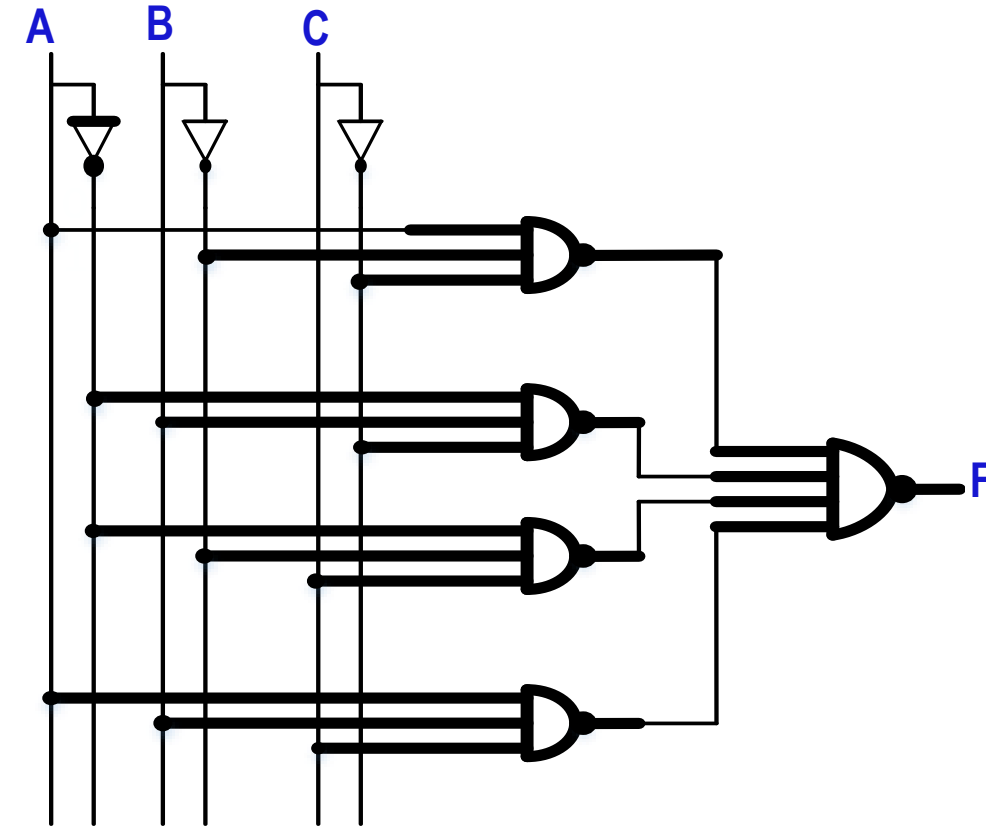
□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

(b) Using Logic gate

Ans.

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$
$$F = A \cdot \overline{B} \cdot \overline{C} \cdot A \cdot B \cdot \overline{C} \cdot A \cdot \overline{B} \cdot C \cdot A \cdot B \cdot C$$



Implementation using Universal Logic Gates



□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

(b) Using Logic gate

Ans.

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$

Draw the logic gates ?

Thanks!